

Senior / Principal Angular + TypeScript + RxJS --- Easy Revision Notes

Part 1 --- JavaScript Fundamentals

Questions 1--35

Revision pattern: Definition → Syntax → Example → Interview point.

The uploaded question bank contains 770 questions across JavaScript, TypeScript, RxJS, Angular, architecture, browser fundamentals, system design and DSA. This document starts the detailed revision notes in the same order as the source.

1. JavaScript Fundamentals

Core JavaScript

1. What are `var` , `let` , and `const` ?

Easy definition

All three declare variables, but they differ mainly in **scope, redeclaration, reassignment, and hoisting behavior**.

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function	Block	Block
Reassign	Yes	Yes	No
Redeclare same scope	Yes	No	No
Hoisted	Yes, initialized as <code>undefined</code>	Yes, but TDZ	Yes, but TDZ
Modern recommendation	Avoid generally	Use when value changes	Default choice

Syntax

```
let count = 10;
count = 20;

const name = "Vivek";
// name = "Raj"; // Error
```

Important: `const` prevents reassignment of the variable, not mutation of an object.

```
const user = { name: "A" };
user.name = "B"; // allowed
// user = {};    // not allowed
```

Interview point: Prefer `const` by default; use `let` when reassignment is required.

2. What is hoisting?

Easy definition

Hoisting means JavaScript processes certain declarations before executing the code in that scope.

```
console.log(x); // undefined
var x = 10;
```

Conceptually:

```
var x;
console.log(x);
x = 10;
```

Function declarations are also available before their textual position:

```
sayHello();

function sayHello() {
  console.log("Hello");
}
```

`let` and `const` are hoisted too, but cannot be accessed before initialization because of the **Temporal Dead Zone**.

3. What is the Temporal Dead Zone?

Easy definition

The TDZ is the period between entering a scope and initializing a `let`, `const`, or `class` declaration.

```
console.log(name); // ReferenceError
let name = "Vivek";
```

The variable exists in the scope, but JavaScript does not allow access to it until the declaration is initialized.

Remember:

```
var    → hoisted + initialized as undefined
let    → hoisted + TDZ
const  → hoisted + TDZ
```

4. What is scope in JavaScript?

Easy definition

Scope determines **where a variable can be accessed**.

Main types:

```
Global scope
Function scope
Block scope
Module scope
```

Example:

```
let globalValue = 1;

function test() {
  let functionValue = 2;
```

```
if (true) {  
  let blockValue = 3;  
  console.log(globalValue, functionValue, blockValue);  
}  
  
// blockValue is not accessible here  
}
```

`var` is function-scoped; `let` and `const` are block-scoped.

5. What is lexical scope?

Easy definition

Lexical scope means a function can access variables based on **where the function was written**, not where it is called.

```
const name = "Vivek";  
  
function outer() {  
  const message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  return inner;  
}  
  
const fn = outer();  
fn(); // Hello
```

`inner()` remembers the lexical environment where it was created.

This is the foundation of **closures**.

6. What is a closure?

Easy definition

A closure happens when a function remembers and can access variables from its outer scope even after the outer function has finished.

```
function counter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const increment = counter();  
  
console.log(increment()); // 1  
console.log(increment()); // 2
```

The returned function still has access to `count` .

Remember:

Closure = `Function` + remembered outer lexical environment

7. Give a real-world use case for closures.

Common uses:

- Data privacy
- Counters
- Function factories
- Memoization
- Callbacks
- Event handlers
- Maintaining state without exposing it publicly

```
function createLogger(prefix) {  
  return function (message) {  
    console.log(`[${prefix}] ${message}`);  
  };  
}  
  
const errorLog = createLogger("ERROR");
```

```
errorLog("API failed");
```

`prefix` is private state captured by the returned function.

8. What is an execution context?

Easy definition

An execution context is the environment JavaScript creates to run code.

Important types:

```
Global Execution Context
Function Execution Context
Eval Execution Context
```

A function execution context contains information needed to execute that function, including:

- Variables
- Function declarations
- Scope information
- `this`
- Access to outer lexical environments

The JavaScript engine manages execution contexts using the **call stack**.

9. What is the call stack?

Easy definition

The call stack is a LIFO (Last In, First Out) structure used to keep track of currently executing functions.

```
function one() {
  two();
}
```

```
function two() {
  three();
}
```

```
}  
  
function three() {  
  console.log("Hello");  
}  
  
one();
```

Conceptually:

```
three()  
two()  
one()  
global
```

When `three()` finishes, it is removed first.

Too much synchronous recursion can cause:

```
RangeError: Maximum call stack size exceeded
```

10. How does JavaScript execute synchronous code?

JavaScript executes synchronous code sequentially on the current execution thread.

Example:

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

A simplified model:

JavaScript **code**



Call Stack



Execute one operation at **a time**

Asynchronous APIs such as timers, network requests and browser events are coordinated by the host environment and Event Loop.

11. Difference between primitive and reference types?

Primitive values represent individual immutable values.

Common primitives:

```
string
number
bigint
boolean
undefined
null
symbol
```

Objects, arrays and functions are objects/reference values.

```
let a = 10;
let b = a;

b = 20;

console.log(a); // 10
```

With an object:

```
const a = { value: 10 };
const b = a;

b.value = 20;

console.log(a.value); // 20
```

Both variables refer to the same object.

Important interview wording: JavaScript variables hold values. For objects, the value is a reference to an object.

12. What is pass-by-value vs pass-by-reference?

JavaScript is fundamentally **pass-by-value**.

For primitive values, the copied value is the primitive itself.

For objects, the copied value is the **reference to the object**.

```
function change(obj) {  
  obj.name = "B";  
}  
  
const user = { name: "A" };  
change(user);  
  
console.log(user.name); // B
```

But replacing the parameter does not replace the caller's variable:

```
function replace(obj) {  
  obj = { name: "B" };  
}  
  
const user = { name: "A" };  
replace(user);  
  
console.log(user.name); // A
```

Remember: JavaScript does not use true pass-by-reference for ordinary function arguments.

13. Shallow copy vs deep copy?

Shallow copy

Copies the top-level structure, but nested objects remain shared.

```
const original = {
  name: "A",
  address: { city: "Nagpur" }
};

const copy = { ...original };

copy.address.city = "Pune";

console.log(original.address.city); // Pune
```

Deep copy

Nested objects are also copied.

Modern simple option:

```
const copy = structuredClone(original);
```

For JSON-compatible data, another older approach is:

```
const copy = JSON.parse(JSON.stringify(original));
```

But JSON cloning loses values such as `Date`, `Map`, `Set`, `undefined`, functions and special object types.

14. What are spread and rest operators?

Both use `...`, but their purpose depends on context.

Spread = expand

```
const a = [1, 2];
const b = [...a, 3];

console.log(b); // [1, 2, 3]
```

```
const user = { name: "A" };
const updated = { ...user, age: 30 };
```

Rest = collect

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
  
sum(1, 2, 3);
```

Memory trick:

```
Spread → unpack  
Rest   → collect
```

15. What is destructuring?

Easy definition

Destructuring extracts values from arrays or properties from objects into variables.

```
const user = {  
  name: "Vivek",  
  age: 30  
};  
  
const { name, age } = user;
```

Array:

```
const numbers = [10, 20];  
  
const [first, second] = numbers;
```

Rename:

```
const { name: userName } = user;
```

Default value:

```
const { role = "user" } = user;
```

Useful in Angular/TypeScript code for extracting API response properties and function arguments.

16. What is `this` in JavaScript?

Easy definition

`this` refers to a context determined mainly by **how a function is called**.

Method call:

```
const user = {  
  name: "Vivek",  
  print() {  
    console.log(this.name);  
  }  
};  
  
user.print(); // Vivek
```

Constructor:

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Vivek");
```

For normal functions, don't decide `this` merely by looking at where the function is declared; inspect the call site.

17. How does `this` behave in arrow functions?

Arrow functions do **not create their own `this`**.

They capture `this` from the surrounding lexical scope.

```
const user = {  
  name: "Vivek",
```

```
regular() {  
  console.log(this.name);  
},  
  
arrow: () => {  
  console.log(this.name);  
}  
};
```

The arrow function does not receive `this` from `user` .

This is especially useful for callbacks:

```
class User {  
  name = "Vivek";  
  
  printLater() {  
    setTimeout(() => {  
      console.log(this.name);  
    }, 1000);  
  }  
}
```

The arrow retains the class method's `this` .

18. What are `call` , `apply` , and `bind` ?

They allow explicit control of `this` .

```
function greet(city) {  
  console.log(this.name, city);  
}  
  
const user = { name: "Vivek" };
```

`call`

Arguments individually:

```
greet.call(user, "Nagpur");
```

apply

Arguments as an array:

```
greet.apply(user, ["Nagpur"]);
```

bind

Returns a new function with fixed `this` :

```
const boundGreet = greet.bind(user);  
boundGreet("Nagpur");
```

Memory:

```
call → invoke now, arguments separately  
apply → invoke now, arguments array  
bind → return new function
```

19. What are prototypes?

Easy definition

A prototype is an object from which another object can inherit properties and methods.

```
const personMethods = {  
  greet() {  
    console.log("Hello");  
  }  
};  
  
const user = Object.create(personMethods);  
  
user.greet();
```

JavaScript's object model is prototype-based.

Classes provide a convenient syntax over this underlying prototype mechanism.

20. What is the prototype chain?

When JavaScript cannot find a property directly on an object, it looks at the object's prototype, then the prototype's prototype, and so on.

```
const user = {};  
  
console.log(user.toString);
```

`toString()` is not an own property of `user`; it is found through the prototype chain.

Conceptually:

```
user  
↓  
Object.prototype  
↓  
null
```

Property lookup stops when the property is found or the chain reaches `null`.

21. How does JavaScript inheritance work?

JavaScript inheritance works through prototypes.

With classes:

```
class Animal {  
  speak() {  
    console.log("Animal sound");  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log("Woof");  
  }  
}  
  
const dog = new Dog();
```

```
dog.speak();  
dog.bark();
```

`Dog.prototype` is connected to `Animal.prototype`.

The class syntax makes inheritance easier to write, but prototype relationships still exist underneath.

22. Class inheritance vs prototype inheritance?

Prototype inheritance

You directly connect objects through prototypes.

```
const animal = {  
  speak() {  
    console.log("sound");  
  }  
};  
  
const dog = Object.create(animal);
```

Class inheritance

You use class syntax:

```
class Dog extends Animal {}
```

Important point:

```
JavaScript's underlying inheritance mechanism → prototypes  
class syntax → cleaner abstraction over prototypes
```

23. What are ES modules?

ES Modules (ESM) are JavaScript's standard module system.

Export:

```
export function add(a, b) {  
  return a + b;  
}
```

```
}  
  
export const PI = 3.14;
```

Import:

```
import { add, PI } from "./math.js";
```

Default export:

```
export default class User {}
```

Import:

```
import User from "./user.js";
```

Benefits:

- Explicit dependencies
- Static structure
- Better tooling
- Tree shaking support
- Encapsulation

24. CommonJS vs ES modules?

Feature CommonJS ES Modules

Export	<code>module.exports</code>	<code>export</code>	Import	<code>require()</code>	<code>import</code>	Common environment	Node.js
historically	Browser + Node.js	Static module syntax	No	Yes	Tree-shaking friendly	Less suitable	Yes

CommonJS:

```
const math = require("./math");  
module.exports = { add };
```

ESM:

```
import { add } from "./math.js";
export { add };
```

Modern frontend applications generally use ESM.

25. Optional chaining and nullish coalescing?

Optional chaining `?.`

Safely accesses a property when an intermediate value may be `null` or `undefined`.

```
const city = user?.address?.city;
```

Instead of manually:

```
const city =
  user && user.address
    ? user.address.city
    : undefined;
```

Nullish coalescing `??`

Uses a fallback only when the left side is `null` or `undefined`.

```
const name = user.name ?? "Guest";
```

Difference from `||`:

```
0 || 10    // 10
0 ?? 10    // 0

"" || "A"  // "A"
"" ?? "A"  // ""
```

Remember:

```
?. → safely access
?? → fallback for null/undefined
```

Array / Object Questions

26. Difference between `map()` and `forEach()` ?

`map()` transforms each item and returns a **new array**.

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(n => n * 2);  
  
console.log(doubled); // [2, 4, 6]
```

`forEach()` executes a function for each item and returns `undefined`.

```
numbers.forEach(n => {  
  console.log(n);  
});
```

Use:

```
map      → need transformed array  
forEach → need side effects
```

27. Difference between `filter()` and `find()` ?

`filter()` returns **all matching elements**.

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.filter(n => n > 2);  
// [3, 4]
```

`find()` returns the **first matching element**.

```
const result = numbers.find(n => n > 2);  
// 3
```

If no match:

```
filter → []  
find   → undefined
```

28. How does `reduce()` work?

`reduce()` combines an array into a single accumulated result.

Syntax:

```
array.reduce((accumulator, currentValue) => {  
  return newAccumulator;  
}, initialValue);
```

Sum:

```
const numbers = [1, 2, 3, 4];  
  
const total = numbers.reduce(  
  (sum, n) => sum + n,  
  0  
);  
  
console.log(total); // 10
```

Grouping:

```
const users = [  
  { name: "A", role: "admin" },  
  { name: "B", role: "user" },  
  { name: "C", role: "admin" }  
];  
  
const grouped = users.reduce((result, user) => {  
  (result[user.role] ??= []).push(user);  
  return result;  
}, {});
```

Remember: `reduce()` is useful when many values need to become one result.

29. `slice()` vs `splice()` ?

`slice()`

Returns a portion without changing the original array.

```
const a = [1, 2, 3, 4];

const result = a.slice(1, 3);

console.log(result); // [2, 3]
console.log(a);      // unchanged
```

`splice()`

Changes the original array.

```
const a = [1, 2, 3, 4];

a.splice(1, 2);

console.log(a); // [1, 4]
```

Memory:

```
slice → copy portion
splice → modify original
```

30. What do `some()` and `every()` do?

`some()` checks whether **at least one** item satisfies a condition.

```
[1, 2, 3].some(n => n > 2); // true
```

`every()` checks whether **all** items satisfy a condition.

```
[1, 2, 3].every(n => n > 0); // true
```

Memory:

some → at least one
every → all

31. How does `sort()` behave?

By default, JavaScript `sort()` converts elements to strings and sorts lexicographically.

```
[10, 2, 5].sort();  
// [10, 2, 5] as string-style ordering can be surprising
```

For numbers, provide a comparator:

```
const numbers = [10, 2, 5];  
  
numbers.sort((a, b) => a - b);  
// [2, 5, 10]
```

Descending:

```
numbers.sort((a, b) => b - a);
```

For objects:

```
users.sort((a, b) => a.age - b.age);
```

Important: `sort()` mutates the array.

For immutable code:

```
const sorted = [...numbers].sort((a, b) => a - b);
```

32. How would you remove duplicates from an array?

For primitive values, `Set` is simple:

```
const numbers = [1, 2, 2, 3, 3];

const unique = [...new Set(numbers)];

console.log(unique); // [1, 2, 3]
```

For objects, define what makes an object unique:

```
const users = [
  { id: 1, name: "A" },
  { id: 2, name: "B" },
  { id: 1, name: "A" }
];

const unique = [
  ...new Map(users.map(user => [user.id, user])).values()
];
```

Here `id` is treated as the uniqueness key.

33. How would you group an array of objects?

Use `reduce()` :

```
const users = [
  { name: "A", department: "IT" },
  { name: "B", department: "HR" },
  { name: "C", department: "IT" }
];

const grouped = users.reduce((result, user) => {
  const key = user.department;

  if (!result[key]) {
    result[key] = [];
  }

  result[key].push(user);

  return result;
}, {});
```

Result:

```
{
  IT: [
    { name: "A", department: "IT" },
    { name: "C", department: "IT" }
  ],
  HR: [
    { name: "B", department: "HR" }
  ]
}
```

Interview point: Always clarify the grouping key and desired output structure.

34. How would you flatten a nested array?

Use `flat()` .

```
const numbers = [1, [2, [3, 4]]];

numbers.flat(2);
// [1, 2, 3, 4]
```

Default depth is 1:

```
[1, [2, 3]].flat();
// [1, 2, 3]
```

For arbitrary nesting:

```
numbers.flat(Infinity);
```

Alternative recursive solution may be useful in coding interviews when built-in methods are disallowed.

35. What is `flatMap()` ?

`flatMap()` performs:

```
map() + one-level flat()
```

Example:

```
const numbers = [1, 2, 3];

const result = numbers.flatMap(n => [n, n * 2]);

console.log(result);
// [1, 2, 2, 4, 3, 6]
```

Equivalent conceptually:

```
numbers
  .map(n => [n, n * 2])
  .flat();
```

Useful when each input item can produce zero, one, or multiple output items.

JavaScript Quick Revision

var	→ function scoped
let	→ block scoped + reassignable
const	→ block scoped + not reassignable
Hoisting	→ declarations processed before execution
TDZ	→ let/const unavailable before initialization
Scope	→ where a variable is accessible
Closure	→ function remembers outer variables
Stack	→ LIFO function execution
Primitive	→ value
Object	→ reference value
Spread	→ expand
Rest	→ collect
Destructure	→ extract values
this	→ depends on invocation for normal functions
Arrow	→ lexical this
call	→ invoke + separate args
apply	→ invoke + array args

bind → return bound function

Prototype → inheritance mechanism
ESM → import/export modules

?.
?? → safe access
?? → null/undefined fallback

map → transform
forEach → side effect
filter → all matches
find → first match
reduce → many values → one result
slice → non-mutating portion
splice → mutates array
some → any?
every → all?
sort → mutates; numeric comparator needed
flat → flatten
flatMap → map + one-level flatten

High-Value Interview Connections

Closure

Closure
↓
Lexical scope
↓
Private **state**
↓
Callbacks
↓
Memoization

this

Normal **function**
↓
Look at how it **is** called

Arrow **function**



Uses surrounding lexical this

Array methods

Need new transformed array	→ map
Need matching items	→ filter
Need first matching item	→ find
Need boolean	→ some/every
Need one accumulated value	→ reduce
Need portion	→ slice
Need modify original	→ splice

Principal/Senior interview expectation

For JavaScript questions, don't stop at the definition. Be ready to explain:

1. What problem does the feature solve?
2. What happens internally?
3. What is the time/space implication?
4. Does it mutate data?
5. What is the modern alternative?
6. Where have you used it in an enterprise application?
7. What common bug can it cause?

How to Use These Notes

These notes continue the uploaded 770-question bank from Q36 onward. Each topic uses: **Easy definition → syntax → example → when to use → interview point → quick revision.**

For questions where the source gives only a question title, the notes explain the concept without changing the source's topic order.

2. JavaScript Async / Event Loop --- Q36--55

36--42. Event Loop, microtasks and macrotasks

Event Loop --- easy definition: JavaScript normally runs synchronous code on a single call stack. The Event Loop coordinates completed asynchronous work with the stack.

```
Call Stack
  ↓
Web/Node APIs
  ↓
Queues
  ↓
Event Loop
  ↓
Call Stack
```

Microtasks: Promise callbacks and other microtask work. They are processed after the current synchronous work completes, before the next task is normally taken.

```
console.log("A");

Promise.resolve().then(() => console.log("C"));

console.log("B");
// A B C
```

Tasks/macrotasks: Examples include timer callbacks such as `setTimeout`.

```
setTimeout(() => console.log("B"), 0);
```

Typical ordering:

1. Synchronous code
2. Microtasks
3. Next task/macrotask

43. Promise

A Promise represents the eventual result of an asynchronous operation.

```
const promise = fetch("/api/users");

promise
  .then(response => response.json())
```

```
.then(data => console.log(data))  
.catch(error => console.error(error));
```

States:

```
pending → fulfilled  
pending → rejected
```

A settled Promise is either fulfilled or rejected.

45--46. async/await

`async` makes a function return a Promise. `await` pauses that async function until the awaited Promise settles; it does not block the JavaScript thread.

```
async function loadUsers() {  
  try {  
    const response = await fetch("/api/users");  
    return await response.json();  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Think:

```
async → Promise-returning function  
await → readable Promise handling
```

47. Promise vs Observable

Promise Observable

Usually one eventual result 0, 1, or many values

Starts immediately when created Usually lazy

Promise has one settlement Observable can emit repeatedly

`.then/.catch/.finally` `.pipe/.subscribe`

No built-in stream composition like Rich operators RxJS

Cancellation requires supporting Unsubscription is part of mechanism Observable model

48. Promise.all()

Runs multiple Promises and fulfills when all fulfill. Rejects when one rejects.

```
const [users, products] = await Promise.all([
  getUsers(),
  getProducts()
]);
```

Use for independent operations where all results are required.

49. Promise.allSettled()

Waits for every Promise and reports each result.

```
const results = await Promise.allSettled([
  getUsers(),
  getProducts()
]);
```

Useful when one failure should not prevent you from seeing other outcomes.

50. Promise.race()

Settles when the first input Promise settles.

```
const result = await Promise.race([
  request(),
  timeout()
]);
```

Useful for timeout patterns.

51. Promise.any()

Fulfills when the first input Promise fulfills. It rejects only if all inputs reject.

Useful when several equivalent sources are available and any successful result is acceptable.

52. Error handling with async/await

```
try {  
  const data = await loadData();  
} catch (error) {  
  console.error(error);  
} finally {  
  hideLoader();  
}
```

53. Unhandled Promise rejection

If a Promise rejects and no handler handles it, the runtime reports an unhandled rejection. In production applications, always establish an intentional error-handling path.

54. Cancelling async operations

Use an API-specific cancellation mechanism such as `AbortController` for fetch.

```
const controller = new AbortController();  
  
fetch("/api/users", {  
  signal: controller.signal  
});  
  
controller.abort();
```

In RxJS, unsubscription is the standard cancellation mechanism.

55. Output question

```
console.log("A");  
  
setTimeout(() => console.log("B"));  
  
Promise.resolve().then(() => console.log("C"));  
  
console.log("D");
```

Typical browser/Node scheduling result:

```
A
D
C
B
```

Why?

```
A, D → synchronous
C     → microtask
B     → timer task
```

Quick revision

```
Sync first
↓
Microtasks
↓
Tasks/macrotasks
```

3. TypeScript Fundamentals --- Q56--97

56. What is TypeScript?

TypeScript is JavaScript with a static type system and additional developer tooling. TypeScript code is transformed into JavaScript for execution.

```
let age: number = 30;
let name: string = "Vivek";
```

57--58. Advantages / disadvantages

Advantages - Earlier error detection - Better IDE autocomplete - Safer refactoring - Interfaces/types - Generics - Stronger contracts

Disadvantages - Additional build step - Type-system learning curve - Types are erased from ordinary runtime JavaScript output - Incorrect type design can add complexity

59. TypeScript → JavaScript

```
.ts / .tsx  
↓  
TypeScript compiler / build tool  
↓  
JavaScript  
↓  
Browser / Node.js
```

Types generally do not exist at runtime after compilation.

60. Static typing

A variable's expected type is checked by the TypeScript compiler.

```
let count: number = 10;  
  
// count = "10"; // compile-time error
```

61--62. Primitive types and inference

```
string  
number  
boolean  
bigint  
symbol  
null  
undefined
```

Type inference means TypeScript can determine a type without an explicit annotation.

```
const name = "Vivek";  
// inferred as string
```

63--68. Interfaces, types and implementation

```
interface User {  
  id: number;
```

```
    name: string;
}
```

Interface extension:

```
interface Admin extends User {
    permissions: string[];
}
```

Type composition:

```
type UserWithRole = User & {
    role: string;
};
```

A class can implement an interface:

```
class UserService implements Service {
    // implementation
}
```

Interface vs type

```
interface → object contracts, declaration merging, extends
type       → aliases, unions, intersections, tuples and more
```

Both can model many object shapes. Choose based on project conventions and the required type-system feature.

69--70. Optional and readonly

```
interface User {
    id: number;
    nickname?: string;
    readonly createdAt: Date;
}
```

Optional means the property may be absent. `readonly` prevents assignment through that type after initialization.

71--75. Arrays, tuples, unions, intersections, literals

```
const names: string[] = ["A", "B"];
const ids: Array<number> = [1, 2];

const user: [number, string] = [1, "Vivek"];

let id: string | number;

type AdminUser = User & {
  permissions: string[];
};

let direction: "up" | "down";
```

76--78. Enums

```
enum Status {
  Pending,
  Success,
  Failed
}
```

String enum:

```
enum Status {
  Pending = "PENDING",
  Success = "SUCCESS",
  Failed = "FAILED"
}
```

For many application-domain states, a literal union is a lightweight alternative:

```
type Status = "PENDING" | "SUCCESS" | "FAILED";
```

79--81. Custom types and assertions

```
type UserId = string;

type User = {
  id: UserId;
```

```
    name: string;
};
```

Type assertion:

```
const value = document.getElementById("app") as HTMLDivElement;
```

Important: type assertion does not convert the runtime value.

```
const value = "123" as unknown as number;
// runtime value is still a string
```

82--85. `unknown` vs `any`

`unknown` means: "I have a value, but I must check it before using it."

```
function print(value: unknown) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  }
}
```

`any` disables most type checking:

```
let value: any = 10;
value.foo.bar(); // compiler cannot protect you
```

Remember:

```
unknown → safe boundary
any      → escape hatch
```

86--88. `void` vs `never`

`void` generally means a function does not return a useful value.

```
function log(message: string): void {
  console.log(message);
}
```

`never` means the function cannot complete normally.

```
function fail(message: string): never {  
    throw new Error(message);  
}
```

Also useful for exhaustive checks.

89. `null` vs `undefined`

`undefined` commonly means a value is absent/uninitialized.

`null` commonly means an explicitly empty value.

With strict null checking, they are distinct types.

90--92. Function typing and async functions

```
const add = (a: number, b: number): number => a + b;  
  
function greet(name: string = "Guest"): string {  
    return `Hello ${name}`;  
}  
  
function findUser(id?: number): void {  
    // ...  
}
```

Async:

```
async function loadUser(): Promise<User> {  
    return await getUser();  
}
```

93--97. tsconfig and strictness

`tsconfig.json` configures the TypeScript project.

Important:

```
{  
    "compilerOptions": {  
        "strict": true,  

```

```
"strictNullChecks": true,  
"noImplicitAny": true  
}  
}
```

strict enables a group of stronger type-checking options.

strictNullChecks prevents treating `null` / `undefined` as automatically valid for every type.

noImplicitAny reports cases where TypeScript would otherwise infer `any` implicitly.

Module resolution describes how TypeScript finds imported modules and their type declarations.

Quick revision

```
interface → object contract  
type      → flexible type composition  
union     → A OR B  
intersection → A AND B  
unknown   → safe unknown value  
any        → disable checking  
void       → no useful return  
never      → cannot normally return  
as         → compile-time assertion, not conversion  
strict     → stronger type safety
```

4. Advanced TypeScript --- Q98--135

98--102. Generics

Easy definition: Generics allow you to write reusable code while preserving the type of the data.

```
function identity<T>(value: T): T {  
    return value;  
}  
  
const n = identity(10);    // number  
const s = identity("hello"); // string
```

Generic interface:

```
interface ApiResponse<T> {  
    data: T;
```

```
    success: boolean;
  }
```

Generic constraint:

```
function getId<T extends { id: number }>(value: T): number {
  return value.id;
}
```

103. keyof

Produces a union of property keys.

```
type User = {
  id: number;
  name: string;
};

type UserKey = keyof User;
// "id" | "name"
```

104. typeof in type context

Gets the type of an existing value.

```
const config = {
  apiUrl: "/api",
  timeout: 5000
};

type Config = typeof config;
```

105. Indexed access types

Get a property's type:

```
type User = {
  id: number;
  name: string;
};
```

```
type UserId = User["id"]; // number
```

106. Conditional types

Choose a type based on a condition.

```
type IsString<T> = T extends string ? true : false;
```

107. Mapped types

Create a new type by transforming each property.

```
type Optional<T> = {  
  [K in keyof T]?: T[K];  
};
```

This is conceptually how utility types such as `Partial<T>` can be built.

108. Template literal types

Build types using string patterns.

```
type EventName = `on${"Click" | "Hover"}`;  
// "onClick" | "onHover"
```

109--116. Type guards and narrowing

A type guard gives TypeScript information that narrows a union.

```
function print(value: string | number) {  
  if (typeof value === "string") {  
    console.log(value.toUpperCase());  
  } else {  
    console.log(value.toFixed(2));  
  }  
}
```

`instanceof` :

```
if (error instanceof Error) {
  console.log(error.message);
}
```

in :

```
if ("email" in value) {
  console.log(value.email);
}
```

User-defined guard:

```
function isUser(value: unknown): value is User {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value
  );
}
```

Discriminated union:

```
type Result =
  | { kind: "success"; data: User }
  | { kind: "error"; message: string };

function handle(result: Result) {
  if (result.kind === "success") {
    return result.data;
  }

  return result.message;
}
```

Exhaustive check:

```
function assertNever(value: never): never {
  throw new Error(`Unexpected value: ${value}`);
}
```

117. as const

Makes literal values readonly and preserves narrow literal types.

```
const roles = ["admin", "user"] as const;  
// readonly ["admin", "user"]
```

118. satisfies

Checks that an expression conforms to a type while retaining the expression's more specific inferred type.

```
type Config = {  
  mode: "dev" | "prod";  
};  
  
const config = {  
  mode: "prod"  
} satisfies Config;
```

119. Structural typing

TypeScript generally checks whether a value has the required structure rather than requiring nominal identity.

```
interface Point {  
  x: number;  
  y: number;  
}  
  
const point = {  
  x: 10,  
  y: 20,  
  label: "A"  
};  
  
const p: Point = point;
```

120--132. Utility types

```
Partial<User>  
Required<User>  
Readonly<User>  
Pick<User, "id" | "name">
```

```
Omit<User, "password">
Record<Role, Permission[]>
Exclude<A, B>
Extract<A, B>
NonNullable<T>
ReturnType<F>
Parameters<F>
Awaited<T>
```

Examples

```
type UpdateUser = Partial<User>;

type UserSummary = Pick<User, "id" | "name">;

type PublicUser = Omit<User, "password">;

type Permissions = Record<"admin" | "user", string[]>;

type A = Exclude<"a" | "b", "a">; // "b"

type B = Extract<"a" | "b", "a" | "c">; // "a"

type C = NonNullable<string | null>; // string

type Fn = (id: number, active: boolean) => User;

type Params = Parameters<Fn>; // [number, boolean]
type Result = ReturnType<Fn>; // User

type Data = Awaited<Promise<User>>; // User
```

133--135. Declaration files, augmentation, type-only imports

`.d.ts` files describe types for JavaScript libraries or ambient APIs.

```
declare module "legacy-lib" {
  export function doSomething(): void;
}
```

Module augmentation extends an existing module's declarations.

Type-only import:

```
import type { User } from "../models";
```

It communicates that the import is needed only for typing and can help keep runtime imports clean.

Quick revision

Generics	→ reusable + type-safe
keyof	→ property keys
typeof	→ type of value
T[K]	→ property type
Conditional	→ type-level <code>if/else</code>
Mapped	→ transform properties
Guard	→ prove/narrow type
as const	→ preserve literals + readonly
satisfies	→ validate shape, retain inference
Utility types	→ reuse type transformations
.d.ts	→ type declarations

5. RxJS Fundamentals --- Q136--155

136. What is RxJS?

RxJS is a library for composing asynchronous and event-based programs using **Observables and operators**.

137. What problem does RxJS solve?

It provides a consistent way to model streams such as:

- HTTP results
- User input
- Router events
- WebSocket messages
- Timers
- Application state

140. Observable

Easy definition: An Observable represents a stream that can emit values over time.

```
const numbers$ = of(1, 2, 3);

numbers$.subscribe(value => {
  console.log(value);
});
```

141--143. Notifications and lazy execution

Observable notifications:

```
next(value)
error(error)
complete()
```

An Observable is commonly lazy: creating it does not necessarily perform its work.

```
const observable$ = new Observable(subscriber => {
  console.log("Work started");

  subscriber.next(1);
  subscriber.complete();
});

console.log("Before");
observable$.subscribe();
```

The work starts when subscribed.

144--150. Creating Observables

```
of(1, 2, 3);
from([1, 2, 3]);
interval(1000);
timer(1000);
defer(() => of(Date.now()));

EMPTY;
NEVER;
throwError(() => new Error("Failed"));
```

`of()` emits supplied values.

from() converts supported inputs such as arrays, iterables and Promises into Observable emissions.

interval() emits sequential numbers periodically.

timer() emits after a delay and can optionally repeat.

defer() creates the Observable at subscription time.

EMPTY completes without emitting.

NEVER neither emits nor completes.

throwError() creates an Observable that errors.

151--153. Subscription

```
const subscription = observable$.subscribe({
  next: value => console.log(value),
  error: error => console.error(error),
  complete: () => console.log("done")
});

subscription.unsubscribe();
```

Unsubscription stops the subscription and runs teardown logic where applicable.

154--155. Operator and pipe

An operator transforms, filters, combines or controls an Observable stream.

```
numbers$.pipe(
  map(n => n * 2),
  filter(n => n > 2)
);
```

pipe() composes operators.

Quick revision

```
Observable → stream
subscribe  → start/listen
next       → value
error      → failure
complete   → finished
```

operator → stream transformation
pipe → combine operators
unsubscribe → stop subscription

6. RxJS Transformation & Filtering --- Q156-170

map()

Transforms every emitted value.

```
source$.pipe(  
  map(user => user.name)  
);
```

RxJS `map()` is conceptually similar to JavaScript array `map()`, but it transforms **Observable emissions over time**.

filter()

Allows only emissions matching a condition.

```
source$.pipe(  
  filter(user => user.active)  
);
```

scan()

Accumulates continuously as values arrive.

```
interval$.pipe(  
  scan((total, value) => total + value, 0)  
);
```

Think:

scan → running accumulation

reduce()

Accumulates and emits the final result when the source completes.

```
source$.pipe(  
  reduce((total, value) => total + value, 0)  
);
```

tap()

Performs a side effect without changing the emitted value.

```
source$.pipe(  
  tap(() => console.log("API started")),  
  map(user => user.name)  
);
```

Do not use `tap()` as the normal place for business transformation. Use `map()` or another appropriate transformation operator.

distinctUntilChanged()

Suppresses consecutive duplicate values.

```
search$.pipe(  
  distinctUntilChanged()  
);
```

It compares the current value with the previously emitted value.

take()

Take a fixed number of emissions.

```
source$.pipe(take(1));
```

takeUntil()

Continue until another Observable emits.

```
source$.pipe(  
  takeUntil(destroy$)  
);
```

takeWhile()

Continue while the predicate is true.

```
source$.pipe(  
  takeWhile(user => user.active)  
);
```

first()

Take the first matching emission and complete. If no matching value is emitted, the operator can error depending on usage.

```
source$.pipe(first());
```

skip()

Ignore the first N emissions.

```
source$.pipe(skip(2));
```

Quick revision

map	→ transform
filter	→ keep matching
scan	→ running accumulator
reduce	→ final accumulator
tap	→ side effect
distinctUntilChanged	→ remove consecutive duplicates
take	→ first N
takeUntil	→ stop when notifier emits
takeWhile	→ continue while condition true
first	→ first matching
skip	→ ignore first N

7. RxJS Higher-Order Mapping --- Q171--187

171. Higher-order Observable

An Observable that emits another Observable.

```
outer$.pipe(  
  map(value => inner$(value))  
);
```

That produces an Observable of Observables.

Higher-order mapping operators flatten/control those inner Observables.

172--174. `switchMap()`

Easy definition: Subscribe to the latest inner Observable and unsubscribe from the previous inner subscription when a new outer value arrives.

```
search$.pipe(  
  switchMap(term => http.get(`/api/search?q=${term}`))  
);
```

Best mental model:

`switchMap` → LATEST

Use cases:

- Autocomplete
- Search
- Route-param-driven data
- Latest selection wins

Cancellation means the previous **inner subscription** is unsubscribed. For Angular HttpClient requests, this commonly results in cancellation of the underlying request.

175--176. `mergeMap()`

Subscribes to multiple inner Observables concurrently.

```
ids$.pipe(  
  mergeMap(id => http.get(`/api/users/${id}`))  
);
```

Think:

`mergeMap` → PARALLEL

Use when every request/result matters and operations are independent.

177--178. `concatMap()`

Queues inner Observables and subscribes to the next only after the previous completes.

```
saveRequests$.pipe(  
  concatMap(request => save(request))  
);
```

Think:

`concatMap` → SEQUENTIAL

Useful when order matters.

179--180. `exhaustMap()`

Ignores new outer emissions while the current inner Observable is running.

```
submitClick$.pipe(  
  exhaustMap(() => saveForm())  
);
```

Think:

`exhaustMap` → IGNORE WHILE BUSY

Excellent for preventing duplicate form submissions.

Comparison

Operator New emission while busy Pattern

`switchMap` Cancel previous Latest `mergeMap` Start another Parallel `concatMap` Queue it Sequential
`exhaustMap` Ignore it Busy lock

185. When NOT to use `switchMap()`

Do not use it when every operation must finish.

Bad example:

```
saveClicks$.pipe(  
  switchMap(() => saveOrder())  
);
```

If another click arrives, the previous save subscription may be cancelled.

Use `concatMap()` when saves must be processed in order, or `mergeMap()` when independent saves may run concurrently.

186--187. Nested subscriptions

Problem:

```
user$.subscribe(user => {  
  orders$(user.id).subscribe(orders => {  
    // nested subscription  
  });  
});
```

Problems:

- Harder error handling
- Harder cancellation
- Subscription management becomes complicated
- Data flow becomes difficult to read

Refactor:

```
user$.pipe(  
  switchMap(user => orders$(user.id))
```

```
).subscribe(orders => {  
  // ...  
});
```

Memorize

```
map          → transform  
switchMap    → latest  
mergeMap     → parallel  
concatMap    → sequential  
exhaustMap   → ignore while busy
```

8. RxJS Combination Operators --- Q188-200

combineLatest()

Combines the latest values from multiple Observables.

It normally waits until **each input has emitted at least once**.

```
combineLatest([name$, age$]).subscribe(([name, age]) => {  
  console.log(name, age);  
});
```

After initial values exist, a new emission from any input can produce a new combined value.

forkJoin()

Waits for all supplied Observables to complete and then emits their **last values**.

```
forkJoin({  
  users: users$,  
  roles: roles$,  
  permissions: permissions$  
}).subscribe(result => {  
  console.log(result.users);  
});
```

Best for parallel HTTP calls where all results are needed once.

If an input never completes, `forkJoin()` cannot produce its final emission. If an input errors and the error is not handled, the combined Observable errors.

forkJoin vs **combineLatest**

```
forkJoin
→ wait for all to complete
→ emit final values
→ common for one-time HTTP calls

combineLatest
→ wait for each to emit once
→ continue reacting to latest values
→ common for reactive UI state
```

zip()

Pairs emissions by position.

```
zip(a$, b$)
```

Think:

```
a1 + b1
a2 + b2
a3 + b3
```

It waits for matching positions.

zip vs **combineLatest**

```
zip          → pair by emission index
combineLatest → combine latest available values
```

withLatestFrom()

The **source** Observable triggers the output and takes the latest value from other Observables.

```
saveClick$.pipe(
  withLatestFrom(formValue$),
```

```
switchMap(([_, form]) => save(form))
);
```

Think:

```
withLatestFrom → source triggers
combineLatest → any input can trigger after initialization
```

merge()

Merges emissions from multiple Observables into one stream.

```
merge(click$, keyboard$)
```

Values arrive as their source emits.

concat()

Subscribes to Observables sequentially.

```
concat(first$, second$)
```

`second$` starts after `first$` completes.

Quick revision

```
combineLatest → latest from all
forkJoin      → final values after all complete
zip           → pair by position
withLatestFrom → source triggers + latest companion
merge         → concurrent emissions into one stream
concat        → sequential Observables
```

9. RxJS Subjects / Multicasting --- Q201--218

Observable vs Subject

Observable: Consumers normally subscribe; the producer logic is represented by the Observable.

Subject: Acts as both an Observable and an Observer. It can receive values through `next()` and multicast them to current subscribers.

```
const subject = new Subject<number>();

subject.subscribe(v => console.log("A", v));
subject.subscribe(v => console.log("B", v));

subject.next(10);
```

Both subscribers receive `10` .

Subject

Does not retain a current value for future subscribers.

```
const subject = new Subject<number>();
subject.next(1);

subject.subscribe(v => console.log(v));
// 1 is not replayed
```

BehaviorSubject

Stores the current/latest value and requires an initial value.

```
const state$ = new BehaviorSubject<number>(0);

state$.subscribe(v => console.log(v));
// immediately receives 0

state$.next(10);
```

ReplaySubject

Replays a configured number/window of previous values.

```
const subject = new ReplaySubject<number>(2);

subject.next(1);
subject.next(2);
subject.next(3);
```

```
subject.subscribe(v => console.log(v));  
// 2, 3
```

AsyncSubject

Emits the **last value only when the Subject completes**.

Comparison

Type Initial value Replays previous values Typical use

	Subject	No Events	BehaviorSubject	Yes Latest	Current state	ReplaySubject	No Configurable
history	Recent history						
AsyncSubject	No						
Final value on complete							
Final result							

asObservable()

Expose a Subject as an Observable so consumers cannot directly call `next()`.

```
private readonly stateSubject =  
    new BehaviorSubject<State>(initialState);  
  
readonly state$ =  
    this.stateSubject.asObservable();
```

This protects the write side.

Hot vs cold

Cold Observable: Each subscriber can get its own execution.

```
const data$ = defer(() => http.get("/api/data"));
```

Hot Observable: The producer exists independently of a particular subscriber.

A Subject is a common hot/multicasting mechanism.

Making a cold stream shared

```
const shared$ = source$.pipe(  
  share()  
);
```

share()

Shares one underlying subscription among multiple subscribers.

shareReplay()

Shares and replays buffered emissions.

```
const users$ = http.get<User[]>("/api/users").pipe(  
  shareReplay({ bufferSize: 1, refCount: true })  
);
```

Useful for sharing a cached result across subscribers.

HTTP caching concept

Without sharing, multiple subscriptions may cause multiple executions depending on the source.

HTTP Observable

↓

shareReplay(1)

↓

Subscriber A

Subscriber B

Subscriber C

The exact cache lifetime and invalidation strategy still need to be designed.

refCount

With ref-counting, the shared connection tracks active subscribers. When the subscriber count reaches zero, the shared subscription can be torn down depending on the operator configuration.

Quick revision

Subject → multicast event

BehaviorSubject → current value

ReplaySubject → previous values

AsyncSubject → final **value on** complete

asObservable() → expose **read-only** Observable API

share() → **share** execution

shareReplay() → **share** + replay

10. RxJS Error Handling --- Q219--230

Handling errors

Use `catchError()` for recovery or transformation of an error.

```
api$.pipe(  
  catchError(error => {  
    console.error(error);  
    return of([]);  
  })  
);
```

What should `catchError()` return?

It must return an Observable.

```
catchError(() => of([]))
```

or rethrow:

```
catchError(error => {  
  return throwError(() => error);  
})
```

`throwError()`

Creates an Observable that errors.

```
throwError(() => new Error("Failed"));
```

`retry()`

Resubscribes after errors.

```
api$.pipe(  
  retry(2)  
);
```

Use carefully. Do not blindly retry operations that are non-idempotent or where retrying cannot reasonably recover.

For example, retrying a failed `POST` that created an order may create duplicates unless the API supports idempotency.

retryWhen()

Allows custom retry timing/conditions.

Conceptually:

```
error  
↓  
retry decision  
↓  
wait  
↓  
resubscribe
```

finalize()

Runs teardown logic when the Observable terminates or the subscription is unsubscribed.

```
loading = true;  
  
api$.pipe(  
  finalize(() => loading = false)  
);
```

catchError vs **finalize**

```
catchError → recover/replace/rethrow an error  
finalize   → cleanup regardless of normal/error/unsubscribe termination
```

API fallback

```
users$.pipe(  
  catchError(() => of(cachedUsers))  
);
```

Global Angular HTTP errors

A common enterprise pattern is a functional interceptor:

```
export const errorInterceptor: HttpInterceptorFn =  
  (req, next) => {  
    return next(req).pipe(  
      catchError(error => {  
        // central logging / selected global handling  
        return throwError(() => error);  
      })  
    );  
  };  
};
```

Do not put every business-specific error decision into a global interceptor; allow feature code to handle errors that need feature-specific behavior.

Quick revision

```
catchError → recover  
throwError → create/rethrow error  
retry       → retry fixed times  
retryWhen   → custom retry policy  
finalize    → cleanup
```

11. RxJS Timing / Search --- Q231--238

debounceTime()

Waits until no new value arrives for the specified duration.

```
search$.pipe(  
  debounceTime(300)  
);
```

Best for search input.

throttleTime()

Allows an emission and suppresses other emissions for a time window.

```
scroll$.pipe(  
  throttleTime(200)  
);
```

Useful when you need regular rate limiting.

auditTime()

Waits for a time window and then emits the most recent value available for that window.

Useful for high-frequency UI events when periodic latest values are sufficient.

Debounce vs throttle

```
debounceTime → wait for quiet period  
throttleTime → limit frequency  
auditTime    → periodically sample latest
```

Autocomplete

```
searchControl.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  filter(term => term.length >= 2),  
  switchMap(term =>  
    this.http.get<Result[]>("/api/search", {  
      params: { q: term }  
    })  
  )  
);
```

Why `switchMap()` ?

User types:

a
an

```
ang  
angu  
angular
```

Only **the** latest search result is relevant.

Previous inner subscriptions are cancelled/unsubscribed when a new term arrives.

Prevent duplicate searches

```
distinctUntilChanged()
```

Cancel old search request

```
switchMap(term => this.search(term))
```

Interview answer

For autocomplete, I normally combine `debounceTime`, `distinctUntilChanged`, validation/filtering, and `switchMap`. Debounce reduces request frequency, `distinctUntilChanged` avoids consecutive duplicates, and `switchMap` ensures stale searches no longer remain active.

Quick revision

```
Search → debounceTime + distinctUntilChanged + switchMap  
Scroll → throttleTime / auditTime depending on requirement
```

12. Angular Fundamentals --- Q239--256

Angular

Angular is a TypeScript-based framework for building web applications with components, templates, dependency injection, routing, forms and other platform capabilities.

SPA

A Single Page Application loads the application shell and then changes views/data without requiring a full browser document reload for every navigation.

Angular vs React

Angular → full framework/**platform**
React → UI **library**/ecosystem

The exact architecture depends on project choices; Angular provides more batteries-included capabilities.

Major Angular building blocks

Components
Templates
Directives
Pipes
Services
Dependency **Injection**
Router
Forms
HttpClient
Signals
RxJS **integration**

Standalone component

A standalone component can declare its own template dependencies without requiring an NgModule declaration.

```
@Component({  
  selector: "app-user",  
  standalone: true,  
  imports: [CommonModule],  
  template: `<p>{{ name }}</p>`  
})  
export class UserComponent {  
  name = "Vivek";  
}
```

Modern Angular emphasizes standalone APIs.

Standalone vs NgModule

NgModule

→ groups declarations/providers/imports

Standalone

→ **dependencies** are declared closer to the component/directive/pipe

Standalone APIs reduce module ceremony and make lazy loading and dependency boundaries clearer.

CLI

Angular CLI provides commands for generating, building, testing, serving and maintaining Angular applications.

angular.json

Workspace/build configuration such as projects, build targets and assets.

main.ts

Application entry point.

A modern standalone application commonly bootstraps with:

```
bootstrapApplication(AppComponent, appConfig);
```

app.config.ts

A common location for application-level providers/configuration.

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes),  
    provideHttpClient()  
  ]  
};
```

bootstrapApplication()

Bootstraps a standalone root component and configures application providers.

Quick revision

```
main.ts           → entry point
bootstrapApplication → start standalone app
app.config.ts     → application providers/config
angular.json      → workspace/build configuration
standalone        → component-level dependency model
```

13. Angular Templates & Binding --- Q257-269

Template syntax

Angular templates combine HTML with Angular binding/control-flow syntax.

Interpolation

```
<h1>{{ title }}</h1>
```

Displays an expression result as text.

Property binding

```
<button [disabled]="isSaving">
  Save
</button>
```

Sets a DOM/component property.

Event binding

```
<button (click)="save()">
  Save
</button>
```

Responds to an event.

Two-way binding

```
<input [(ngModel)]="name">
```

Conceptually combines:

```
[property]  
+  
(event)
```

For a custom component, modern Angular can use `model()` for a component model.

One-way data flow

A common architecture is:

```
Parent state  
  ↓  
Child input  
  ↓  
User event  
  ↓  
Parent/state update  
  ↓  
New input
```

This makes data flow easier to reason about.

Template reference variable

```
<input #nameInput>  
  
<button (click)="nameInput.focus()">  
  Focus  
</button>
```

as syntax

Can capture an expression/result in a template variable in supported template constructs.

Optional chaining / safe navigation

```
{{ user?.address?.city }}
```

Prevents errors when an intermediate value is nullish.

[property] vs interpolation

Use property binding when setting an actual property:

```
<img [src]="imageUrl">
```

Interpolation is primarily text interpolation:

```
<p>{{ imageUrl }}</p>
```

Although Angular can interpolate in some attribute/property contexts, explicit property binding communicates intent more clearly.

(event) vs **[...]**

```
(event)    → listen  
[property] → provide value  
[(...)]    → two-way binding syntax
```

14. Angular Components & Communication -- - Q270--285

Parent → child

Traditional:

```
@Input() user!: User;
```

Modern signal input:

```
user = input.required<User>();
```

Parent:

```
<app-user [user]="selectedUser" />
```

Child → parent

Traditional:

```
@Output() saved = new EventEmitter<User>();
```

Modern output:

```
saved = output<User>();
```

Emit:

```
this.saved.emit(user);
```

Sibling / unrelated components

Common choices:

```
Parent-mediated communication  
Shared service  
Signal-based service state  
NgRx/store  
Router state
```

Choose according to scope and complexity.

input()

Creates a signal input.

```
name = input<string>("");
```

Required:

```
user = input.required<User>();
```

output()

Creates a component output.

```
selected = output<User>();
```

model()

Creates a writable model input that supports component two-way binding patterns.

```
value = model<string>("");
```

Parent can use:

```
<app-input [(value)]= "name" />
```

Input transforms

Transforms input values at the component boundary.

Conceptually:

```
disabled = input(false, {  
  transform: booleanAttribute  
});
```

Shared service

For feature-local shared state:

```
@Injectable()  
export class CartState {  
  private itemsSubject =  
    new BehaviorSubject<Item[]>([]);
```

```
items$ = this.itemsSubject.asObservable();  
}
```

Service vs NgRx

Use a service when state is relatively local/simple and does not require a large event/state architecture.

NgRx becomes more useful when you need:

- Complex shared state
- Explicit actions
- Reducers/effects/selectors
- Strong event traceability
- Multiple feature consumers
- Predictable state transitions

Quick revision

```
Parent → child      → input()  
Child → parent      → output()  
Two-way component  → model()  
Feature shared state → service  
Complex global state → NgRx
```

15. Angular Lifecycle --- Q286--297

Lifecycle order --- practical view

Common hooks:

```
constructor  
↓  
ngOnChanges  
↓  
ngOnInit  
↓  
ngDoCheck  
↓  
ngAfterContentInit  
↓
```

```
ngAfterContentChecked
```

```
↓
```

```
ngAfterViewInit
```

```
↓
```

```
ngAfterViewChecked
```

```
↓
```

```
ngOnDestroy
```

Exact invocation depends on inputs/content/view and change-detection cycles.

ngOnChanges()

Runs when Angular detects changes to data-bound inputs.

```
ngOnChanges(changes: SimpleChanges) {  
  console.log(changes);  
}
```

ngOnInit()

Runs once after initial input processing.

Good for initialization that depends on Angular-bound inputs being available.

ngDoCheck()

Custom change-detection hook. Use sparingly because it can run frequently.

Content hooks

`ngAfterContentInit()` and `ngAfterContentChecked()` relate to projected content (`ng-content`).

View hooks

`ngAfterViewInit()` and `ngAfterViewChecked()` relate to the component's view and view children.

ngOnDestroy()

Runs before destruction. Use it for cleanup where required.

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
}
```

```
}
```

Modern Angular also provides other lifecycle-aware utilities such as `DestroyRef` and `takeUntilDestroyed()`.

Constructor vs `ngOnInit()`

```
constructor → class construction + DI
ngOnInit    → Angular initialization phase
```

Avoid putting substantial Angular initialization logic in the constructor.

Heavy lifecycle logic

Hooks can execute repeatedly depending on the hook. Keep them small and move reusable/business logic into suitable services, computed state, or explicit methods.

Quick revision

```
constructor → object construction
OnChanges   → input changes
OnInit      → initial setup
DoCheck      → custom checking; expensive if abused
AfterContent → projected content
AfterView    → component view
OnDestroy   → cleanup
```

16. Angular Dependency Injection --- Q298-316

Dependency Injection

Easy definition: Instead of a class creating its dependencies itself, Angular provides them.

```
@Injectable({ providedIn: "root" })
export class UserService {}
```

```
constructor(private userService: UserService) {}
```

or modern:

```
private userService = inject(UserService);
```

Why DI?

- Loose coupling
- Testability
- Reusable services
- Configurable implementations
- Lifecycle management

Injector hierarchy

Angular can have providers at different levels. A component-level provider creates a provider instance associated with that component injector and its descendants unless overridden.

```
@Component({  
  providers: [FeatureService]  
})
```

Two components can therefore have separate `FeatureService` instances.

providedIn: 'root'

```
@Injectable({  
  providedIn: "root"  
})
```

Registers the service with the application/root injector and enables tree-shaking when appropriate.

EnvironmentInjector

Provides injectable dependencies in environment-level contexts such as application/route/dynamic-component scenarios.

ElementInjector

Associated with elements/components/directives and their provider configuration.

InjectionToken

Used when the dependency is not naturally represented by a class, such as configuration values.

```
export const API_URL =  
  new InjectionToken<string>("API_URL");
```

Provider:

```
{  
  provide: API_URL,  
  useValue: "/api"  
}
```

Provider strategies

```
{ provide: Token, useClass: MyService }  
{ provide: Token, useValue: config }  
{ provide: Token, useFactory: createService }  
{ provide: Token, useExisting: ExistingToken }
```

```
useClass    → instantiate implementation  
useValue    → use supplied value  
useFactory  → create using factory function  
useExisting → alias existing provider
```

DI modifiers

```
@Optional() → don't fail if unavailable  
@Self()     → look only in current injector  
@SkipSelf() → start from parent  
@Host()     → constrain lookup around host boundary
```

Modern `inject()` :

```
private router = inject(Router);
```

Constructor injection vs `inject()`

Both retrieve dependencies from Angular DI. `inject()` is especially useful in field initializers and functional APIs such as guards/interceptors.

Quick revision

```
DI → Angular supplies dependencies
root → application-level provider
component provider → scoped instance
InjectionToken → non-class dependency
useClass/useValue/useFactory/useExisting → provider strategies
```

17. Angular Directives --- Q317--330

Directive

A directive adds behavior to an existing DOM element or Angular template.

Component = directive + template.

Attribute directive

Changes behavior/appearance of an existing element.

```
@Directive({
  selector: "[appHighlight]"
})
export class HighlightDirective {}
```

Structural directive

Changes the structure of rendered content.

Historically:

```
<div *ngIf="isVisible">Hello</div>
```

The `*` syntax expands into an embedded template representation.

ElementRef

Provides access to the underlying element. Avoid direct DOM manipulation when Angular APIs can do the job.

Renderer2

Provides an Angular abstraction for DOM operations.

TemplateRef

Represents an Angular template that can be instantiated.

ViewContainerRef

Represents a location where embedded views/components can be inserted.

HostListener

```
@HostListener("mouseenter")
onEnter() {
  // ...
}
```

HostBinding

Binds a host element property/class/attribute.

```
@HostBinding("class.active")
isActive = true;
```

Directive Composition API

Allows reusable directive behavior to be composed onto a component/directive rather than manually duplicating behavior.

Background directive example

```
@Directive({
  selector: "[appHighlight]"
})
export class HighlightDirective {
  private el = inject(ElementRef);
  private renderer = inject(Renderer2);
```

```

@HostListener("mouseenter")
onEnter() {
  this.renderer.setStyle(
    this.el.nativeElement,
    "background",
    "yellow"
  );
}

@HostListener("mouseleave")
onLeave() {
  this.renderer.removeStyle(
    this.el.nativeElement,
    "background"
  );
}
}

```

Tooltip directive

A production tooltip should consider:

- Positioning
- Keyboard accessibility
- Focus
- Escape key
- Cleanup
- View encapsulation
- Overlay boundaries
- Screen readers

For a platform component library, keep tooltip behavior reusable and configurable.

Quick revision

```

Component → directive with template
Attribute directive → behavior/appearance
Structural directive → view structure
ElementRef → element reference
Renderer2 → DOM abstraction
TemplateRef → template
ViewContainerRef → insertion point

```

18--20. Angular Template Primitives, Dynamic Components & Pipes --- Q331--365

Template primitives

ng-container

Groups Angular template logic without adding a real DOM element.

```
<ng-container *ngIf="user">
  {{ user.name }}
</ng-container>
```

ng-template

Defines a template that is not rendered immediately.

```
<ng-template #loading>
  Loading...
</ng-template>
```

ng-content

Projects parent-provided content into a component.

```
<div class="card">
  <ng-content></ng-content>
</div>
```

ngTemplateOutlet

Renders a `TemplateRef`.

```
<ng-container
  *ngTemplateOutlet="template">
</ng-container>
```

Content projection

Parent:

```
<app-card>
  <button>Save</button>
</app-card>
```

Child:

```
<ng-content></ng-content>
```

Multi-slot:

```
<ng-content select="[header]"></ng-content>
<ng-content select="[body]"></ng-content>
```

ContentChild vs ViewChild

```
ContentChild → projected content
ViewChild    → component's own view
```

View queries and dynamic components

Traditional:

```
@ViewChild(UserComponent)
userComponent!: UserComponent;
```

Modern signal query:

```
userComponent = viewChild(UserComponent);
```

Multiple:

```
users = viewChildren(UserComponent);
```

Dynamic component:

```
const ref = viewContainerRef.createComponent(UserComponent);
```

Pass input:

```
ref.setInput("user", user);
```

Listen to output:

```
ref.instance.saved.subscribe(user => {  
  // ...  
});
```

`ComponentRef` represents the dynamically created component instance and its associated view.

For advanced dynamic creation, `EnvironmentInjector` can supply the correct dependency environment.

Pipes

A pipe transforms display data in templates.

```
{{ amount | currency }}
```

Custom:

```
@Pipe({  
  name: "initials",  
  pure: true  
})  
export class InitialsPipe implements PipeTransform {  
  transform(name: string): string {  
    return name  
      .split(" ")  
      .map(x => x[0])  
      .join("");  
  }  
}
```

Pure pipe: Angular can avoid rerunning it when input references/primitive values have not changed.

Impure pipe: Can run much more frequently during change detection and can therefore be expensive.

AsyncPipe

```
<div *ngIf="user$ | async as user">
  {{ user.name }}
</div>
```

It subscribes to supported asynchronous sources and handles subscription cleanup when the view is destroyed.

If the source errors, the AsyncPipe does not silently convert the error into a value. Handle the error upstream when needed:

```
userVm$ = user$.pipe(
  catchError(() => of(null))
);
```

An initial asynchronous stream may have no value available yet, so design templates for loading/null states.

Quick revision

```
ng-container → no extra DOM
ng-template → deferred/template definition
ng-content → content projection
TemplateOutlet → render TemplateRef
ViewChild → own view
ContentChild → projected content
Pure pipe → input-change based
Impure pipe → frequent execution risk
AsyncPipe → subscribe + cleanup
```

21. Angular Forms --- Q366--384

Template-driven vs reactive forms

```
Template-driven → template-centric, simpler forms
```

Reactive → model-centric, **explicit** form model

For complex enterprise forms, reactive forms are often easier to compose, validate, test and generate dynamically.

FormControl

Represents one form value and its state.

```
name = new FormControl("");
```

FormGroup

Groups controls.

```
form = new FormGroup({  
  name: new FormControl(""),  
  email: new FormControl("")  
});
```

FormArray

Dynamic list of controls/groups.

```
items = new FormArray([  
  new FormControl("")  
]);
```

FormBuilder

Reduces form creation boilerplate.

```
form = this.fb.group({  
  name: ['', Validators.required],  
  age: [0, Validators.min(18)]  
});
```

Validators

```
Validators.required
Validators.minLength(3)
Validators.maxLength(100)
Validators.email
Validators.min(18)
Validators.max(100)
```

Custom validator

```
function adult(control: AbstractControl) {
  return control.value >= 18
    ? null
    : { adult: true };
}
```

Async validator

Returns an asynchronous validation result.

```
function uniqueEmail(
  service: UserService
): AsyncValidatorFn {
  return control =>
    service.isEmailTaken(control.value).pipe(
      map(taken => taken ? { taken: true } : null)
    );
}
```

Cross-field validation

Attach validator to the group:

```
const passwordMatch: ValidatorFn =
  group => {
    const password = group.get("password")?.value;
    const confirm = group.get("confirm")?.value;

    return password === confirm
      ? null
      : { passwordMismatch: true };
  };
```

setValue() vs patchValue()

setValue → expects the complete matching structure
patchValue → updates only supplied fields

reset

```
form.reset();
```

Resets values/state according to the form configuration and supplied reset values.

Form state

dirty → value changed
pristine → not changed
touched → control has been focused/blurred
untouched → not touched
valid → passes validation
invalid → validation errors exist
pending → async validation is in progress

valueChanges / statusChanges

```
form.get("name")?.valueChanges.subscribe(value => {  
  console.log(value);  
});
```

Dynamic forms

Use metadata to create controls:

```
fields.forEach(field => {  
  group.addControl(  
    field.name,  
    new FormControl(field.defaultValue)  
  );  
});
```

This is particularly relevant to metadata-driven/low-code applications.

Disable control

```
control.disable();  
control.enable();
```

Conditional validation

```
if (requiresTaxId) {  
  taxId.addValidators(Validators.required);  
} else {  
  taxId.removeValidators(Validators.required);  
}  
  
taxId.updateValueAndValidity();
```

Quick revision

```
FormControl → one control  
FormGroup → object/group  
FormArray → dynamic list  
Validator → synchronous rule  
AsyncValidator → asynchronous rule  
setValue → complete structure  
patchValue → partial update  
valueChanges → values  
statusChanges → validation state
```

22. ControlValueAccessor --- Q385--394

What is ControlValueAccessor?

CVA is Angular's interface for making a custom component behave like a normal form control.

Use it when building reusable controls such as:

```
Custom dropdown  
Date picker  
Rich text editor  
OTP input
```

Custom select
Metadata-driven form field

Core methods

```
interface ControlValueAccessor {  
  writeValue(obj: any): void;  
  registerOnChange(fn: any): void;  
  registerOnTouched(fn: any): void;  
  setDisabledState?(isDisabled: boolean): void;  
}
```

Meaning

`writeValue()` → Angular → component
`registerOnChange()` → component tells Angular about value changes
`registerOnTouched()` → component tells Angular it was touched
`setDisabledState()` → Angular tells component enabled/disabled state

Basic custom control

```
@Component({  
  selector: "app-rating",  
  template: `  
    <button  
      type="button"  
      [disabled]="disabled"  
      (click)="select(1)">  
      1  
    </button>  
  `,  
})  
export class RatingComponent  
  implements ControlValueAccessor {  
  
  value = 0;  
  disabled = false;  
  
  private onChange = (_: number) => {};  
  private onTouch = () => {};  
  
  writeValue(value: number): void {  
    this.value = value ?? 0;  
  }  
}
```

```

registerOnChange(fn: (value: number) => void): void {
  this.onChange = fn;
}

registerOnTouched(fn: () => void): void {
  this.onTouched = fn;
}

setDisabledState(disabled: boolean): void {
  this.disabled = disabled;
}

select(value: number) {
  if (this.disabled) return;

  this.value = value;
  this.onChange(value);
  this.onTouched();
}
}

```

Provide it as a value accessor using Angular's forms integration configuration.

Interview point: CVA is different from ordinary `@Input/@Output` . Input/output is component communication; CVA integrates a component with Angular Forms APIs.

Quick revision

```

writeValue      → write from form
onChange        → component value → form
onTouched       → interaction → form
setDisabledState → disabled state

```

23. Angular Routing --- Q395--419

Router

Angular Router maps URLs to components/views.

```

export const routes: Routes = [
  {
    path: "users",
    component: UsersComponent
  }
]

```

```
}  
];
```

Router outlet

```
<router-outlet />
```

The routed component is rendered there.

routerLink

```
<a [routerLink]="['/users', user.id]">  
  User  
</a>
```

ActivatedRoute

Provides information about the current route.

```
const route = inject(ActivatedRoute);  
  
route.paramMap.subscribe(params => {  
  const id = params.get("id");  
});
```

Router

Programmatic navigation:

```
router.navigate(["/users", id]);
```

Route params

```
/users/123
```

Route:

```
{ path: "users/:id", component: UserComponent }
```

Query params

```
/users?role=admin&page=2
```

Fragment

```
/users#permissions
```

Child routes

```
{
  path: "admin",
  component: AdminComponent,
  children: [
    { path: "users", component: UsersComponent }
  ]
}
```

Lazy loading

Load code only when a route is needed.

```
{
  path: "admin",
  loadChildren: () =>
    import("./admin/admin.routes")
      .then(m => m.ADMIN_ROUTES)
}
```

loadComponent()

Lazy loads a standalone component.

```
{
  path: "settings",
  loadComponent: () =>
    import("./settings.component")
}
```

```
    .then(m => m.SettingsComponent)  
  }
```

loadChildren()

Lazy loads child route definitions/modules.

Route-level DI

Routes can define providers that are scoped to the route/environment.

Guards

Guards control whether navigation can proceed.

Common guards:

```
canActivate  
canActivateChild  
canDeactivate  
canMatch
```

canMatch vs canActivate

```
canMatch    → can this route match/load?  
canActivate → route matched; can navigation activate it?
```

`canMatch` can be useful for conditional route selection and can prevent a lazy route from being matched.

Resolver

Prepares data before route activation.

Route data

Static metadata:

```
{  
  path: "admin",  
  component: AdminComponent,  
}
```

```
data: { title: "Administration" }  
}
```

Navigation events

Router emits events such as navigation start/end/cancel/error.

Preloading

After initial loading, Angular can preload lazy routes according to a configured strategy.

Protect admin route

Use authentication/authorization state in a guard:

```
export const adminGuard: CanActivateFn = () => {  
  const auth = inject(AuthService);  
  const router = inject(Router);  
  
  return auth.hasPermission("ADMIN")  
    ? true  
    : router.parseUrl("/forbidden");  
};
```

Important: Route guards improve frontend navigation control; they are not a substitute for backend authorization.

Quick revision

```
router-outlet → render route  
routerLink    → declarative navigation  
ActivatedRoute → current route info  
Router        → programmatic navigation  
loadComponent → lazy component  
loadChildren  → lazy routes  
canMatch      → route matching  
canActivate   → activation  
resolver      → route data
```

24. Angular HTTP --- Q420--435

HttpClient

Angular's `HttpClient` provides typed HTTP request APIs.

```
private http = inject(HttpClient);

users$ = this.http.get<User[]>("/api/users");
```

HTTP methods

```
http.get<User[]>("/api/users");

http.post<User>("/api/users", user);

http.put<User>(`/api/users/${id}`, user);

http.patch<User>(`/api/users/${id}`, {
  name: "Updated"
});

http.delete<void>(`/api/users/${id}`);
```

Typed response

```
this.http.get<User>("/api/user/1");
```

The generic provides compile-time typing; it does not validate the server response at runtime.

Query params

```
const params = new HttpParams()
  .set("page", 1)
  .set("size", 20);

this.http.get<User[]>("/api/users", { params });
```

Headers

```
const headers = new HttpHeaders({
  "X-Correlation-ID": requestId
});
```

```
this.http.get("/api/users", { headers });
```

HttpContext

Carries request-specific context for interceptors/HTTP behavior.

Conceptual example:

```
const context = new HttpContext()  
  .set(SKIP_AUTH, true);  
  
this.http.get("/api/public", { context });
```

Interceptor

Intercepts HTTP requests/responses centrally.

Modern functional interceptor:

```
export const authInterceptor: HttpInterceptorFn =  
  (req, next) => {  
    const token = inject(AuthService).token();  
  
    const authReq = token  
      ? req.clone({  
        setHeaders: {  
          Authorization: `Bearer ${token}`  
        }  
      })  
      : req;  
  
    return next(authReq);  
  };
```

Functional vs class interceptor

Functional interceptors are concise and integrate naturally with modern provider configuration. Class-based interceptors remain an established pattern.

Loading indicator

A common design:

```
request starts → increment active requests  
request ends   → decrement  
active = 0      → hide loader
```

Use `finalize()` to guarantee decrement/cleanup.

Retry

```
http.get("/api/data").pipe(  
  retry(2)  
);
```

Use a deliberate policy: status codes, idempotency, backoff and maximum attempts.

Request caching

Use a shared Observable/cache layer rather than indiscriminately caching all requests.

```
users$ = this.http.get<User[]>("/api/users").pipe(  
  shareReplay({ bufferSize: 1, refCount: true })  
);
```

Design cache invalidation explicitly.

File upload

```
const formData = new FormData();  
formData.append("file", file);  
  
this.http.post("/api/upload", formData);
```

Progress

```
this.http.post("/api/upload", formData, {  
  observe: "events",  
  reportProgress: true  
});
```

Process upload progress events.

Quick revision

```
HttpClient → HTTP
Interceptor → cross-cutting HTTP behavior
HttpContext → request metadata
HttpParams → query parameters
HttpHeaders → headers
shareReplay → shared cached stream pattern
finalize → loader cleanup
```

25. Authentication & Security --- Q436--452

Authentication vs authorization

```
Authentication → Who are you?
Authorization → What are you allowed to do?
```

JWT

A JSON Web Token commonly carries claims used by a server to identify/authenticate a request.

Typical structure:

```
header.payload.signature
```

Do not treat JWT payload data as secret merely because it is encoded.

Access vs refresh token

```
Access token → short-lived API authorization
Refresh token → obtain a new access token
```

The exact flow depends on the authentication architecture.

HttpOnly cookie

A cookie marked `HttpOnly` cannot be read by normal client-side JavaScript.

This can reduce token exposure to JavaScript-based theft, but does not eliminate all web security risks.

SameSite

Controls when cookies are sent in cross-site requests.

```
Strict
Lax
None
```

`SameSite=None` requires `Secure` .

CORS

Cross-Origin Resource Sharing is a browser security mechanism controlling whether a web page may access resources from another origin.

CORS is enforced by the browser; it is not a replacement for authentication/authorization.

CSRF

Cross-Site Request Forgery tricks a user's browser into sending an authenticated request to a site.

Mitigations depend on architecture and can include:

```
SameSite cookies
CSRF tokens
Origin/Referer validation
Proper request design
```

XSS

Cross-Site Scripting occurs when attacker-controlled script content executes in a user's browser in the security context of the application.

Angular XSS protection

Angular normally treats template-bound values as data and sanitizes certain security-sensitive contexts.

Avoid bypassing Angular's security mechanisms unless the value is fully trusted and the security implications are understood.

DomSanitizer

Provides APIs for working with security-sensitive values. Bypass APIs should be used very carefully.

innerHTML

Directly assigning untrusted HTML can create XSS risk.

Prefer:

```
<div>{{ userContent }}</div>
```

rather than injecting untrusted HTML.

RBAC

Role-Based Access Control:

```
User
  ↓
Roles
  ↓
Permissions
```

Example:

```
type Permission =
  | "USER_READ"
  | "USER_CREATE"
  | "USER_DELETE";
```

Frontend can hide/disable UI and protect routes, but backend must enforce authorization.

Critical interview point

Angular route guards do not secure the backend.

A malicious user can bypass frontend code and call APIs directly. The server must independently verify authentication and authorization.

Quick revision

```
AuthN → identity
AuthZ → permissions
JWT → token format
HttpOnly → JS cannot read cookie
CORS → browser cross-origin policy
CSRF → forged authenticated request
XSS → injected script execution
RBAC → roles → permissions
Backend → final authorization boundary
```

26. Change Detection --- Q453--467

Change detection

Angular checks whether component/template-bound state has changed and updates the DOM as needed.

Default strategy

Angular's default strategy checks components as part of normal change-detection processing.

OnPush

`OnPush` narrows when Angular needs to check a component.

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
```

Common triggers include:

- New input reference/value as applicable
- Events handled in the component/view
- Explicit marking
- Signal changes read by the template
- Other Angular/framework mechanisms that notify the view

Why OnPush?

It reduces unnecessary checking, especially in large component trees.

Signals

When a template reads a signal, Angular tracks that reactive dependency and can mark the relevant view when the signal changes.

ChangeDetectorRef

```
cdr.markForCheck();  
cdr.detectChanges();  
cdr.detach();  
cdr.reattach();
```

markForCheck() marks a component/view so it can be checked in a future change-detection cycle.

detectChanges() immediately checks that view/subtree.

detach() removes a view from the regular change-detection tree.

reattach() adds it back.

ExpressionChangedAfterItHasBeenCheckedError

Indicates Angular detected a bound value changing after it had already checked it in a development-mode check scenario.

Typical solution is not "just add setTimeout"; instead identify the incorrect lifecycle/data-flow timing and move the state update to the correct phase.

Immutable updates

OnPush works naturally with reference-based change detection.

Bad:

```
this.items.push(item);
```

Better:

```
this.items = [...this.items, item];
```

For object updates:

```
this.user = {  
  ...this.user,  
  name: "New"  
};
```

Thousands of rows

Use:

```
OnPush  
track / stable identity  
virtual scrolling when appropriate  
pagination/server-side querying  
avoid expensive template expressions  
memoized/computed derived state  
efficient data structures
```

Quick revision

```
Default → broader checking  
OnPush → narrower checking  
markForCheck → schedule checking  
detectChanges → check now  
detach → stop regular checking  
Signals → dependency-aware view updates
```

27. Angular Signals --- Q468--487

Signal

A signal is a reactive value that Angular can track.

```
count = signal(0);  
  
count.set(1);  
count.update(value => value + 1);  
  
console.log(count());
```

Writable vs readonly

```
const writable = signal(0);  
const readonly = writable.asReadonly();
```

Consumers can read `readonly()` but should not update it directly.

`computed()`

Creates derived state.

```
firstName = signal("Vivek");  
lastName = signal("Gangotri");  
  
fullName = computed(() =>  
  `${firstName()} ${lastName()}`  
);
```

It is read as:

```
fullName();
```

`effect()`

Runs side-effect code when its signal dependencies change.

```
effect(() => {  
  console.log("Count:", count());  
});
```

Do not use effects as a general replacement for computed state. If a value can be derived, prefer `computed()`.

`set()` vs `update()`

```
count.set(10);  
  
count.update(value => value + 1);
```

```
set    → provide new value
update → derive new value from previous
```

Dependency tracking

Angular tracks signals read during reactive computations such as `computed()` and effects. When dependencies change, the derived computation/effect can react.

Signal inputs

```
user = input.required<User>();
```

Read:

```
user()
```

Signal queries

```
child = viewChild(ChildComponent);
children = viewChildren(ChildComponent);
```

Signals vs RxJS

Signals

- synchronous reactive **state**
- excellent **for** local/derived UI **state**

RxJS

- asynchronous/event streams
- rich stream operators
- HTTP/event/WebSocket composition

NgRx

- structured application **state**/event architecture

They are complementary rather than mutually exclusive.

Signals vs BehaviorSubject

BehaviorSubject → Observable + current value + stream operators
Signal → reactive value + dependency tracking

toSignal()

Converts an Observable to a Signal.

```
users = toSignal(users$, {  
  initialValue: []  
});
```

toObservable()

Converts a Signal to an Observable.

```
users$ = toObservable(users);
```

Quick revision

```
signal()      → writable reactive value  
computed()    → derived state  
effect()      → side effect  
asReadonly()  → read-only signal  
input()       → signal input  
viewChild()   → signal query  
toSignal()    → Observable → Signal  
toObservable() → Signal → Observable
```

Decision rule

```
Local UI state      → Signal  
Derived state       → computed  
Async stream        → RxJS  
Complex global state → NgRx
```

28--30. Modern Angular Control Flow, @defer, Zone & Rendering --- Q488--523

Control flow

Modern Angular control flow uses:

```
@if (user) {  
  <p>{{ user.name }}</p>  
} @else {  
  <p>Login</p>  
}
```

Loop:

```
@for (user of users; track user.id) {  
  <p>{{ user.name }}</p>  
} @empty {  
  <p>No users</p>  
}
```

Context variables:

```
$index  
$first  
$last  
$even  
$odd
```

Switch:

```
@switch (status) {  
  @case ("loading") {  
    Loading...  
  }  
  @case ("success") {  
    Success  
  }  
  @default {  
    Unknown  
  }  
}
```

track

Tells Angular how to identify items across iterations.

```
@for (user of users; track user.id) {  
  ...  
}
```

Stable identity reduces unnecessary DOM work when collections change.

@defer

Defers loading/rendering of eligible template dependencies until a trigger condition occurs.

```
@defer (on viewport) {  
  <app-heavy-report />  
} @placeholder {  
  <p>Report placeholder</p>  
} @loading {  
  <p>Loading report...</p>  
} @error {  
  <p>Failed to load report.</p>  
}
```

Triggers include:

```
on viewport  
on interaction  
on hover  
on idle  
on immediate  
when
```

Prefetching can load deferred dependencies before the actual render trigger, depending on configuration.

Use `@defer` for heavy, non-critical UI.

Do not use it blindly for content required immediately for the initial experience.

Zone.js

Historically, Angular used Zone.js to know when asynchronous work might require change detection.

`NgZone` provides APIs for working with Angular's zone.

```
ngZone.runOutsideAngular(() => {  
  // high-frequency work  
});
```

Use `runOutsideAngular()` for work that does not need to trigger Angular view updates on every event, such as certain high-frequency browser operations.

Zoneless Angular

Modern Angular can operate without relying on Zone.js for automatic change-detection triggering. Signals and explicit Angular notification mechanisms provide more targeted reactivity.

Rendering pipeline

Simplified:

```
Application state changes  
  ↓  
Angular determines affected views  
  ↓  
Template expressions evaluated  
  ↓  
DOM updates  
  ↓  
Browser rendering
```

Do not confuse Angular change detection with the browser's layout/paint pipeline.

Quick revision

<code>@if</code>	→ conditional
<code>@for</code>	→ loop
<code>track</code>	→ identity
<code>@empty</code>	→ empty state
<code>@switch</code>	→ switch
<code>@defer</code>	→ defer non-critical UI
Zone.js	→ async change-detection coordination
Zoneless	→ explicit/reactive notification model

31. Angular Performance --- Q524--543

Performance checklist

1. Measure first
2. Reduce initial JavaScript
3. Lazy load
4. Defer non-critical UI
5. Use OnPush appropriately
6. Use signals/computed state appropriately
7. Track list identity
8. Virtualize very large lists
9. Avoid expensive template work
10. Optimize images/assets
11. Move CPU-heavy work to Web Workers
12. Consider SSR where it improves the relevant user experience

Avoid functions in templates

```
<!-- potentially evaluated repeatedly -->
{{ calculateTotal() }}
```

Prefer stable derived state/computed values when appropriate.

Memoization

Store a computed result and reuse it for the same inputs.

```
function memoize(fn) {
  const cache = new Map();

  return (key) => {
    if (cache.has(key)) return cache.get(key);

    const value = fn(key);
    cache.set(key, value);
    return value;
  };
}
```

Large lists

Use:

```
@for + track  
OnPush  
pagination  
virtual scrolling  
server-side filtering  
efficient item components
```

Virtual scrolling

Render only the visible portion of a large list rather than creating DOM nodes for every record.

Bundle size

Use:

```
lazy routes  
loadComponent/loadChildren  
@defer  
tree shaking  
code splitting  
remove unused dependencies  
analyze bundles
```

Tree shaking

Build tooling removes unused statically analyzable code from production bundles.

Code splitting

Breaks application code into independently loadable chunks.

Images

Use appropriately sized/compressed images, modern formats where supported, lazy loading for non-critical images, and responsive image strategies.

Identify bottlenecks

Use:

```
Angular DevTools  
Browser Performance panel
```

```
Network panel
Bundle analyzer/build statistics
Lighthouse/Core Web Vitals
Application profiling
```

Core Web Vitals

Important user-experience metrics include loading performance, responsiveness and visual stability. Use current metric definitions/tools when evaluating a production application.

Web Workers

Use for CPU-heavy work that would otherwise block the main thread.

Examples:

```
large data transformation
complex calculations
client-side report generation
parsing large datasets
```

SSR

SSR can improve initial content availability and SEO for appropriate applications, but adds server-side complexity and is not automatically faster for every application.

Quick revision

```
Initial load → lazy/defer/split
Rendering    → OnPush/signals/track
Huge lists   → virtual scroll/pagination
CPU-heavy    → Web Worker
Measure      → DevTools + browser performance + Web Vitals
```

32. Angular SSR / Hydration --- Q544--554

SSR

Server-Side Rendering generates the initial HTML on the server.

Browser request



Angular server



HTML



Browser



Hydration

CSR vs SSR

CSR → browser builds initial UI

SSR → server produces initial HTML

SSR can help:

- Initial content visibility
- Search engine discoverability
- Perceived initial load for suitable pages

Costs:

- Server infrastructure
- More complex rendering environment
- Data-fetching considerations
- Browser/server API differences

SSG

Static Site Generation produces pages ahead of time rather than rendering them per request.

SSR → render per request

SSG → pre-**generate**

CSR → render primarily in browser

Hydration

Hydration connects Angular behavior to server-rendered HTML so the browser can take over without unnecessarily recreating the entire DOM.

Event replay

Events occurring before hydration can be captured/replayed so user interaction is not lost, depending on the configured Angular hydration/event-replay features.

Browser-only APIs

Avoid directly using:

```
window
document
localStorage
```

in code that executes during server rendering.

Use platform checks or appropriate Angular abstractions:

```
if (isPlatformBrowser(platformId)) {
  // browser-only code
}
```

SEO

SSR can make meaningful page content available in server-rendered HTML, which can support search indexing. SEO still depends on routing, metadata, content quality, crawlability and other factors.

When avoid SSR

If the application is highly internal, browser-only, and SEO/initial server-rendered content provides little value, SSR complexity may not be justified.

Quick revision

```
SSR      → HTML on server
CSR      → UI primarily in browser
SSG      → pre-generated HTML
Hydration → activate server HTML
Browser APIs → protect from server execution
```

33. Angular Testing --- Q555--570

TestBed

Angular's testing utility for configuring an Angular testing environment.

```
TestBed.configureTestingModule({  
  providers: [UserService]  
});
```

ComponentFixture

Represents a component under test and provides access to its instance/view.

```
const fixture = TestBed.createComponent(UserComponent);  
const component = fixture.componentInstance;
```

DebugElement

Provides Angular-oriented access to the tested DOM and directives.

Component test

```
it("creates", () => {  
  const fixture =  
    TestBed.createComponent(UserComponent);  
  
  expect(fixture.componentInstance).toBeTruthy();  
});
```

Service test

```
const service =  
  TestBed.inject(UserService);
```

Pipe

Instantiate/test transformation behavior directly or through Angular testing utilities.

Directive

Create a host test component and verify DOM behavior.

Mocking

Provide a fake:

```
const mockAuth = {
  isLoggedIn: () => true
};

 TestBed.configureTestingModule({
  providers: [
    { provide: AuthService, useValue: mockAuth }
  ]
});
```

Spies

Track calls and optionally control return values.

fakeAsync / tick

Useful for deterministic testing of timer-based asynchronous behavior.

```
it("updates later", fakeAsync(() => {
  service.startTimer();

  tick(1000);

  expect(...).toBe(...);
}));
```

HTTP testing

Modern Angular provides HTTP testing utilities that allow requests to be intercepted and expected without calling the real backend.

Conceptually:

```
call service
↓
expect request
↓
flush mock response
```

↓
assert result

Routing tests

Test route configuration/navigation behavior using Angular's router testing utilities appropriate to the Angular version.

Observable testing

Options include:

fakeAsync
RxJS TestScheduler/marble tests
controlled Subjects

Unit vs integration

Unit test
→ one class/function/pipe/directive in isolation

Integration test
→ multiple real application pieces working together

Good test suites focus on behavior rather than implementation details.

Quick revision

TestBed	→ testing environment
Fixture	→ component + view
DebugElement	→ Angular DOM abstraction
Spy	→ observe/mock calls
fakeAsync/tick	→ deterministic async timing
HTTP test	→ mock backend requests

34. NgRx --- Q571--594

NgRx

NgRx is an Angular-oriented reactive state-management ecosystem built around explicit state, actions, reducers, selectors and effects.

Store

Central state container.

```
Component
  ↓ dispatch
Action
  ↓
Reducer
  ↓
State
  ↓
Selector
  ↓
Component
```

Action

Describes an event/intention.

```
export const loadUsers = createAction(
  "[Users Page] Load Users"
);
```

Reducer

Purely calculates new state from previous state + action.

```
export const reducer = createReducer(
  initialState,

  on(loadUsersSuccess, (state, { users }) => ({
    ...state,
    users,
    loading: false
  })))
);
```

Selector

Reads/derives state.

```
export const selectUsers =
  createSelector(
    selectUsersState,
    state => state.users
  );
```

Selectors can be memoized.

Effect

Handles side effects such as HTTP.

```
loadUsers$ = createEffect(() =>
  this.actions$.pipe(
    ofType(loadUsers),
    switchMap(() =>
      this.api.getUsers().pipe(
        map(users => loadUsersSuccess({ users })),
        catchError(error =>
          of(loadUsersFailure({ error }))
        )
      )
    )
  )
);
```

Feature Store

A slice of state dedicated to a feature.

NgRx Entity

Provides normalized entity-state utilities for collections.

Typical conceptual structure:

```
ids
entities
```

This makes CRUD operations and lookup efficient and standardized.

Selector memoization

Selectors can avoid recalculating derived data when relevant inputs have not changed.

Reducers must be pure

A reducer should:

```
same input → predictable output
no HTTP
no random side effect
no mutation of existing state
```

What belongs in reducer?

State transitions and synchronous state calculation.

What belongs in effect?

Side effects:

```
HTTP
navigation
logging/integration effects
external systems
```

Why no API calls in reducer?

Reducers must remain pure, predictable and synchronous.

Optimistic vs pessimistic update

Optimistic

```
Update UI/state first
↓
API request
↓
rollback if failure
```

Pessimistic

```
API request
↓
success
↓
update state
```

Facade

Provides a feature-oriented API between components and state management.

```
users$ = this.store.select(selectUsers);

load() {
  this.store.dispatch(loadUsers());
}
```

Components use:

```
facade.load();
facade.users$;
```

This reduces direct Store knowledge in UI components.

NgRx vs BehaviorSubject vs Signals

```
BehaviorSubject → simple service state/stream
Signal           → local/reactive synchronous state
NgRx             → complex shared state/event architecture
```

Large-app structure

A feature can be organized around:

```
feature/
  data-access/
  state/
  ui/
  pages/
  models/
```

Exact folder structure should follow the team's architecture and boundaries rather than becoming ceremony.

Quick revision

```
Action    → what happened / intention
Reducer   → state transition
Selector  → read/derive
Effect    → side effect
Entity    → normalized collections
Facade    → simpler feature API
```

35. Angular Architecture --- Q595--616

Large Angular application

Prefer feature-oriented boundaries.

```
app/
  core/
  shared/
  features/
    users/
    reports/
    billing/
```

Core

Singleton/global infrastructure:

```
authentication infrastructure
global configuration
interceptors
application services
logging
```

Shared

Reusable UI/utilities that do not belong to one business feature.

```
buttons
form controls
pipes
directives
generic utilities
```

Avoid turning Shared into a dumping ground.

Feature

Business capability and its local UI/state/data access.

```
users/
reports/
orders/
```

Smart vs presentational

```
Container/smart → orchestration/state
Presentational → inputs/outputs + UI
```

This is a useful design technique, not an absolute rule.

Facade pattern

Hide internal state implementation.

```
Component
  ↓
Facade
  ↓
NgRx / service / API
```

Repository pattern

Abstracts data access behind a domain-oriented interface.

```
interface UserRepository {
  getById(id: string): Observable<User>;
}
```

Where API calls live?

Usually in dedicated data-access/API services, repositories or effects depending on the state architecture.

Where business logic lives?

Domain/business logic should not be concentrated in templates. Place it in appropriate domain services, state logic, facades, pure functions and backend services according to ownership.

Where should state live?

Use the narrowest scope that satisfies the requirement.

```
Component-local → component/signal  
Feature shared  → service/store  
Global shared   → application store/state  
Server state    → data-access/cache strategy
```

Prevent huge components

Extract:

```
presentational components  
feature services  
facades  
pure utility functions  
form models  
state management
```

Feature boundaries

A feature boundary should represent a coherent business capability and limit direct dependencies on unrelated features.

Reusable component library

Enterprise component libraries should define:

```
consistent API  
accessibility  
keyboard behavior
```

```
forms integration/CVA
theming
design tokens
testing
documentation
versioning
backward compatibility
```

Cross-feature dependencies

Prefer shared abstractions or events/contracts over direct deep imports between unrelated features.

Authentication architecture

```
Login
↓
server authentication
↓
session/token mechanism
↓
auth state
↓
HTTP authentication
↓
route/UI authorization
```

Authorization

Model permissions explicitly:

```
User → roles → permissions
```

Use frontend checks for UX and backend checks for security.

Global error handling

Combine:

```
HTTP interceptor
central logging/observability
feature-level error handling
user-friendly error presentation
```

Application configuration

Use strongly typed configuration and environment/deployment configuration boundaries. Do not scatter URLs/secrets throughout components.

Principal-level principle

```
Component = UI
Service    = reusable behavior/domain interaction
Facade     = feature API
Store      = state/event architecture
Repository = data-access abstraction
Interceptor = cross-cutting HTTP behavior
```

36. Dynamic / Plugin Architecture --- Q617-625

Dynamic component loading

```
const ref =
  viewContainerRef.createComponent(MyComponent);

ref.setInput("config", config);
```

Plugin system

A scalable plugin architecture needs:

```
Plugin contract
Registry
Metadata
Loader
Isolation boundary
Version compatibility
Failure handling
Security rules
```

Conceptual:

```
Plugin metadata
  ↓
Registry
  ↓
Resolver
  ↓
Dynamic component
  ↓
Inputs/events
```

Metadata-driven UI

Example metadata:

```
{
  "type": "text",
  "name": "firstName",
  "label": "First Name",
  "required": true
}
```

Map metadata to trusted component types:

```
const registry = {
  text: TextFieldComponent,
  select: SelectComponent,
  date: DateFieldComponent
};
```

Then resolve:

```
const component =
  registry[metadata.type];
```

Do not instantiate arbitrary classes or execute arbitrary code from untrusted metadata.

Dynamic inputs/events

```
ref.setInput("config", metadata);

ref.instance.changed.subscribe(value => {
```

```
// handle event  
});
```

Failure isolation

A platform should isolate failures using:

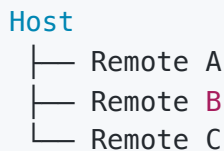
```
error boundaries/defensive loading  
timeouts where appropriate  
fallback UI  
logging  
plugin version validation  
remote failure isolation
```

This is especially relevant to low-code/platform architecture interviews.

37. Micro Frontends / Module Federation --- Q626--644

Microfrontend

A microfrontend divides a frontend into independently owned/deployed application areas.



Why use it?

Potential reasons:

- Independent team ownership
- Independent deployment
- Domain boundaries
- Large organization scaling

Costs:

- Runtime integration complexity

- Dependency/version management
- Shared state complexity
- UX consistency
- Debugging/deployment complexity

Module Federation

Module Federation allows an application to consume modules exposed by another separately built application at runtime.

Typical terms:

```
Host    → consumes remote modules
Remote  → exposes modules
Shared  → dependencies shared between builds
```

Shared dependencies

Common candidates:

```
Angular
RxJS
design system packages
shared contracts
```

Be deliberate about singleton/version compatibility.

Authentication

Avoid making every remote independently invent authentication. A host/shared authentication contract can provide the authenticated context while backend APIs remain authoritative.

Communication

Options:

```
Custom DOM/browser events
Shared event bus
Router/navigation
Shared library contracts
Host-mediated communication
```

Prefer explicit, minimal contracts.

Global store?

Do not automatically create one global store for every microfrontend. Strong coupling through a global store can undermine independent ownership.

Use shared state only when the domain truly requires it.

Routing

Choose clear ownership:

```
Host → top-level route/domain
Remote → internal feature routes
```

Version mismatch

Manage through:

```
compatible dependency versions
peer/shared dependency rules
contract testing
versioned APIs
controlled rollout
```

Independent deployment

A remote can be built/deployed separately while the host resolves the remote at runtime, subject to the chosen deployment architecture.

Remote failure

Design fallback behavior:

```
remote unavailable
↓
host remains usable
↓
show feature fallback
↓
log failure
```

Microfrontend vs modular monolith

Modular monolith

- one deployable application
- **strong** internal module **boundaries**

Microfrontend

- independently deployable frontend units
- runtime/integration **boundary**

Do not introduce microfrontends merely because the application is large.

Quick revision

- Host → consumes
- Remote → exposes
- Shared → common dependency
- Goal → independent ownership/deployment
- Risk → distributed frontend complexity

38. Browser Internals --- Q645--658

Browser rendering

Simplified:

```
HTML
↓
DOM

CSS
↓
CSSOM

DOM + CSSOM
↓
Render tree
↓
Layout
↓
Paint
```

DOM

The browser's object representation of the document structure.

CSSOM

Object representation of CSS rules/styles used by the browser.

Render tree

Represents renderable content used for layout/painting.

Layout / reflow

Calculates geometry/positions/sizes.

Repaint

Updates visual pixels without necessarily recalculating layout.

Compositing

Combines painted layers into the final displayed image.

Reflow vs repaint

```
Layout/reflow → geometry  
Repaint       → pixels
```

A layout-triggering change can be more expensive because it can affect subsequent calculations.

Critical Rendering Path

The sequence through which the browser turns resources into pixels. Optimizing critical CSS, scripts, fonts, images and HTML can improve initial rendering.

requestAnimationFrame()

Schedules work before the browser's next repaint.

```
requestAnimationFrame(() => {  
  element.style.transform = "translateX(100px)";  
});
```

Useful for visual animation/update coordination.

Main thread vs Web Worker

```
Main thread → DOM/UI/event processing  
Worker      → background JS computation
```

Workers cannot directly manipulate the DOM.

What blocks main thread?

```
long JS loops  
large synchronous JSON processing  
expensive calculations  
large DOM operations
```

Event Loop + rendering

A simplified browser model processes tasks/microtasks and rendering opportunities. Microtasks can delay rendering if code continually schedules more microtasks.

Quick revision

```
DOM → HTML structure  
CSSOM → CSS structure  
Layout → geometry  
Paint → pixels  
Composite → layers  
rAF → animation timing  
Worker → CPU work off main thread
```

39--41. Web Fundamentals, Offline-First & API Integration --- Q659--708

What happens when entering a URL?

Simplified:

```
URL
↓
DNS
↓
TCP/TLS as applicable
↓
HTTP request
↓
Server/CDN
↓
HTTP response
↓
Browser parses
↓
DOM/CSSOM
↓
Render
```

Actual behavior can include caching, connection reuse, proxies, service workers and many other optimizations.

HTTP / HTTPS

HTTP is an application protocol for request/response communication.

HTTPS = HTTP over TLS.

TLS provides encryption and server authentication properties.

Methods

```
GET    → retrieve
POST   → create/process
PUT    → replace
PATCH → partial update
DELETE → remove
```

GET vs POST

GET is generally used for retrieval and is intended to be safe/idempotent under HTTP semantics.

POST commonly creates or triggers processing and is not inherently idempotent.

PUT vs PATCH

PUT → replacement semantics
PATCH → partial modification

Status codes

2xx → success
3xx → redirection
4xx → client/request problem
5xx → server-side failure

401 vs 403

401 → authentication required/invalid
403 → understood but **not** permitted

400 vs 422

400 → malformed/invalid request in a general sense
422 → request syntactically understood but semantic validation failed

Exact API conventions can vary.

500 vs 503

500 → internal server error
503 → service unavailable/temporarily unable **to** handle request

Headers

Carry metadata:

Authorization
Content-Type
Accept

Cache-Control
ETag

Cookies

Small pieces of state associated with a domain/path and automatically sent according to cookie rules.

localStorage vs sessionStorage

localStorage → persists across browser sessions
sessionStorage → tied to browser tab/session

Both are synchronous Web Storage APIs and store strings.

IndexedDB

Asynchronous browser database for larger structured client-side data.

localStorage → simple key/value strings
IndexedDB → structured database/object stores/indexes

Service Worker

A background browser worker that can intercept requests and enable capabilities such as caching, offline behavior and push-related functionality.

Cache API

Provides programmatic request/response caching, commonly used with Service Workers.

WebSocket vs SSE

WebSocket → two-way persistent communication
SSE → server-to-browser event stream

Use WebSocket when the client also needs to send real-time messages over the same connection.
SSE is useful for server-to-client event streams.

CDN

Content Delivery Network distributes content closer to users to reduce network latency and origin load.

Browser caching

Browsers can reuse cached resources according to HTTP caching headers.

ETag

A validator identifying a particular representation version.

Conditional request:

```
If-None-Match: "abc"
```

Server may return:

```
304 Not Modified
```

Cache-Control

Controls caching behavior.

Examples:

```
Cache-Control: max-age=3600  
Cache-Control: no-store
```

Offline-first / IndexedDB

IndexedDB

Browser database designed for storing structured data asynchronously.

Concepts:

```
Database  
↓  
Object store
```

↓
Records
↓
Indexes
↓
Transactions

Object store

Comparable conceptually to a collection/table-like storage area for objects.

Index

Provides efficient lookup by a property.

Transaction

Groups database operations with consistency rules.

Offline-first architecture

UI
↓
Local state/cache
↓
IndexedDB
↓
Sync queue
↓
Backend

Reads can work locally. Writes can be queued and synchronized later.

Synchronization

Maintain:

local entity
operation/outbox
server acknowledgement
sync status

Conflict resolution

Possible strategies:

```
last-write-wins
server-authoritative
field-level merge
domain-specific conflict rules
manual resolution
```

Choose based on business requirements.

Optimistic synchronization

Update local state immediately, then synchronize with server.

If synchronization fails:

```
retry
rollback
mark conflict
```

Stale data

Track freshness:

```
updatedAt
version
ETag
sync status
TTL
```

API integration

Pagination

Prefer server-side pagination for large datasets.

```
GET /users?page=2&size=50
```

Response may include:

```
{
  "items": [],
  "page": 2,
  "size": 50,
  "total": 10000
}
```

Server-side filtering

```
GET /users?status=active&department=IT
```

Sorting

```
GET /users?sort=name&direction=asc
```

Search

Combine:

```
debounce
distinctUntilChanged
switchMap
server-side query
pagination
```

API timeout

Use an explicit timeout strategy appropriate to the application.

```
api$.pipe(
  timeout(5000)
);
```

Then handle the timeout error deliberately.

Partial API failure

For independent data, consider:

```
forkJoin + per-request catchError
```

or `combineLatest` /other composition depending on stream semantics.

Example:

```
forkJoin({  
  users: users$.pipe(catchError(() => of([]))),  
  roles: roles$.pipe(catchError(() => of([])))  
});
```

Idempotency

An operation is idempotent if repeating it has the same intended effect as performing it once.

HTTP semantics generally treat GET, PUT and DELETE as idempotent methods, while POST is not inherently idempotent.

For critical create/payment/order APIs, an idempotency key can prevent duplicate processing:

```
Idempotency-Key: unique-request-id
```

API versioning

Common strategies:

```
URL: /api/v1/users  
Header/media type
```

Choose a strategy and establish compatibility/deprecation rules.

Consistent error model

A useful API error contract:

```
{  
  "code": "USER_NOT_FOUND",  
  "message": "User not found",  
  "details": {},
```

```
"correlationId": "..."  
}
```

Frontend can map stable error codes to user-facing behavior.

Quick revision

```
GET    → retrieve  
POST   → create/process  
PUT    → replace  
PATCH → partial update  
401    → authentication  
403    → authorization  
4xx    → client/request  
5xx    → server  
ETag   → representation validator  
IndexedDB → structured offline storage  
WebSocket → two-way real-time  
SSE     → server → client events  
CDN     → distributed content delivery
```

42. Principal-Level System Design --- Q709- -732

709. Enterprise Angular application for millions of users

Think in layers:

```
Browser  
↓  
CDN / Edge  
↓  
Angular application  
↓  
API Gateway  
↓  
Backend services  
↓  
Caches / databases / queues
```

Frontend concerns:

```
feature boundaries
lazy loading
@defer
OnPush/signals
server-side pagination
caching
observability
security
error handling
accessibility
```

710. Scalable dashboard

Avoid loading every widget/data source immediately.

```
Dashboard shell
↓
critical widgets
↓
deferred widgets
↓
parallel data loading
↓
widget-level error handling
```

Use independent loading/error states where appropriate.

711--712. Low-code / metadata-driven UI

Architecture:

```
JSON metadata
↓
schema validation
↓
component registry
↓
renderer
↓
component inputs
↓
event/action dispatcher
↓
services/API
```

Important concerns:

```
schema versioning
security
validation
component compatibility
performance
caching
extensibility
```

713. Dynamic form builder

Metadata:

```
{
  "fields": [
    {
      "type": "text",
      "name": "firstName",
      "validators": ["required"]
    }
  ]
}
```

Renderer maps metadata to CVA-compatible components and constructs a Reactive Form model.

714. Role/permission management

```
User
↓
Roles
↓
Permissions
↓
Frontend capability checks
↓
Backend authorization
```

715. Notification system

Possible architecture:

```
Event producer
↓
Message broker
↓
Notification service
↓
WebSocket/SSE/push
↓
Angular client
```

Persist notifications so users can retrieve unread/history state.

716. Offline-first application

```
UI
↓
Signal/store
↓
IndexedDB
↓
Outbox
↓
Sync worker/service
↓
API
```

Handle:

```
retries
conflicts
versions
connectivity
partial synchronization
```

717. Reporting application

For large reports:

```
request report
↓
backend job
↓
queue/worker
```

```
↓  
object storage  
↓  
download
```

Do not generate enormous reports synchronously in the browser unless the dataset is appropriate.

718. Multi-tenant application

Tenant context must be enforced by backend authorization/data isolation.

Frontend:

```
tenant context  
↓  
configuration  
↓  
feature flags  
↓  
UI
```

Never rely on a client-side tenant ID as the security boundary.

719. Microfrontend platform

Define:

```
host contract  
remote contract  
shared dependencies  
auth contract  
routing contract  
design system  
observability  
failure isolation  
deployment model
```

720--724. Auth, RBAC, caching, errors, observability

Use:

```
Auth → centralized authentication/session mechanism  
RBAC → explicit permissions + backend enforcement
```

Cache → TTL/invalidation/ETag/share strategy
Errors → correlation IDs + centralized logging + user-safe messages
Observability → logs + metrics + traces + frontend telemetry

725. Reduce initial bundle

```
lazy routes
@defer
tree shaking
code splitting
remove unused dependencies
optimize images
analyze chunks
load critical resources first
```

726. NgModule → standalone migration

Do it incrementally:

```
identify feature
↓
convert components/directives/pipes
↓
replace module imports with direct imports
↓
migrate routes/providers
↓
test
↓
repeat
```

Avoid a single risky rewrite when the application is large.

727. Older Angular → modern Angular

Plan:

```
upgrade one major version at a time where required
↓
resolve breaking changes
↓
update dependencies
↓
```

```
adopt standalone
↓
adopt modern control flow
↓
adopt signals selectively
↓
evaluate zoneless/modern rendering
```

Do not modernize everything simultaneously without measuring risk.

728. Signals vs RxJS vs NgRx

```
Signal → local synchronous reactive state
RxJS   → async/event streams
NgRx   → complex shared application state
```

729. Components/services/state

```
Component → presentation/orchestration
Service    → reusable/domain/application behavior
State      → shared reactive data/state transitions
```

730. Millions of records

Never render millions of DOM elements.

Use:

```
server-side pagination
filtering
sorting
virtual scrolling
cursor pagination where suitable
aggregation
progressive loading
```

731. Independent team ownership

Use:

```
business feature boundaries
clear contracts
```

shared design system
API contracts
ownership
versioning
CI/CD
observability

732. When to use microfrontends

Evaluate:

team boundaries
deployment independence
organizational scale
domain autonomy
runtime integration cost
shared dependency complexity

A modular monolith may satisfy many large applications with less operational complexity.

Principal interview framework

Requirements
↓
Scale
↓
Boundaries
↓
Data flow
↓
State
↓
Performance
↓
Security
↓
Failure modes
↓
Observability
↓
Trade-offs

43. Design Patterns & SOLID --- Q733--748

Singleton

One shared instance.

Angular's root-provided services commonly behave as application-level shared services within the relevant injector scope.

Factory

Creates objects without exposing the creation logic to the caller.

```
function createLogger(type: "console" | "remote") {  
  if (type === "console") {  
    return new ConsoleLogger();  
  }  
  
  return new RemoteLogger();  
}
```

Strategy

Encapsulates interchangeable algorithms.

```
interface PricingStrategy {  
  calculate(amount: number): number;  
}
```

Different implementations can be selected at runtime.

Adapter

Converts one interface into another expected interface.

Useful when integrating legacy/external APIs.

Repository

Abstracts data access:

```
Domain  
↓  
Repository interface
```

↓
HTTP / IndexedDB / backend

Facade

Provides a simpler API over a complex subsystem.

Component

↓

Facade

↓

Store + API + transformations

Observer

One-to-many notification relationship.

Angular/RxJS naturally uses this concept:

Observable

↓

Subscribers

Dependency Inversion

High-level code should depend on abstractions rather than concrete implementations.

SOLID

S --- Single Responsibility

A class/module should have one coherent reason to change.

O --- Open/Closed

Open for extension, closed for unnecessary modification.

L --- Liskov Substitution

Subtypes should be usable wherever their base abstraction is expected without breaking correctness.

I --- Interface Segregation

Prefer focused interfaces over huge interfaces.

D --- Dependency Inversion

Depend on abstractions, not concrete implementation details.

Angular + SOLID

DI → supports dependency inversion
Services → can separate responsibilities
Components → presentation responsibility
Interfaces → contracts
Facades → simplify complex subsystems

Quick revision

Factory → create
Strategy → interchangeable behavior
Adapter → convert **interface**
Repository → data access
Facade → simplify subsystem
Observer → notify subscribers
SOLID → maintainable design principles

44. DSA / Coding --- Q749--770

Big O

Big O describes how time/space usage grows as input size grows.

Common complexities:

$O(1)$ → constant
 $O(\log n)$ → logarithmic
 $O(n)$ → linear
 $O(n \log n)$ → common efficient sorting
 $O(n^2)$ → quadratic

750. Two Sum

Goal: find two values adding to target.

Optimal common approach: Hash Map.

```
function twoSum(nums, target) {  
  const seen = new Map();  
  
  for (let i = 0; i < nums.length; i++) {  
    const needed = target - nums[i];  
  
    if (seen.has(needed)) {  
      return [seen.get(needed), i];  
    }  
  
    seen.set(nums[i], i);  
  }  
  
  return [];  
}
```

Complexity:

```
Time → O(n)  
Space → O(n)
```

751. Find duplicates

```
const seen = new Set();  
const duplicates = new Set();  
  
for (const value of nums) {  
  if (seen.has(value)) {  
    duplicates.add(value);  
  } else {  
    seen.add(value);  
  }  
}
```

752. First non-repeating character

```
function firstUniqueChar(str) {  
  const counts = new Map();  
  
  for (const ch of str) {  
    counts.set(ch, (counts.get(ch) ?? 0) + 1);  
  }  
}
```

```
}

for (const ch of str) {
  if (counts.get(ch) === 1) {
    return ch;
  }
}

return null;
}
```

753. Reverse string

```
const reversed = str.split('').reverse().join('');
```

Interview alternative:

```
let result = '';

for (let i = str.length - 1; i >= 0; i--) {
  result += str[i];
}
```

754. Palindrome

```
function isPalindrome(str) {
  return str === str.split('').reverse().join('');
}
```

755. Largest/smallest

```
const max = Math.max(...numbers);
const min = Math.min(...numbers);
```

For huge arrays, prefer a loop to avoid argument-list limitations.

756. Second largest

One-pass approach:

```
function secondLargest(nums) {
  let largest = -Infinity;
  let second = -Infinity;

  for (const n of nums) {
    if (n > largest) {
      second = largest;
      largest = n;
    } else if (n > second && n !== largest) {
      second = n;
    }
  }

  return second;
}
```

Clarify whether duplicate values count as the same value.

757. Remove duplicates

```
const unique = [...new Set(nums)];
```

758. Merge sorted arrays

Two-pointer approach:

```
function merge(a, b) {
  const result = [];
  let i = 0;
  let j = 0;

  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) {
      result.push(a[i++]);
    } else {
      result.push(b[j++]);
    }
  }

  return result
    .concat(a.slice(i))
    .concat(b.slice(j));
}
```

Time:

$O(n + m)$

759. Binary search

Requires sorted input.

```
function binarySearch(nums, target) {
  let left = 0;
  let right = nums.length - 1;

  while (left <= right) {
    const mid =
      left + Math.floor((right - left) / 2);

    if (nums[mid] === target) return mid;

    if (nums[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return -1;
}
```

Time:

$O(\log n)$

760. Implement debounce

```
function debounce(fn, delay) {
  let timer;

  return function (...args) {
    clearTimeout(timer);

    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}
```

```
};  
}
```

761. Implement throttle

```
function throttle(fn, delay) {  
  let last = 0;  
  
  return function (...args) {  
    const now = Date.now();  
  
    if (now - last >= delay) {  
      last = now;  
      fn.apply(this, args);  
    }  
  };  
}
```

Clarify leading/trailing behavior in a real implementation.

762. Implement map

```
Array.prototype.myMap = function (callback) {  
  const result = [];  
  
  for (let i = 0; i < this.length; i++) {  
    result.push(  
      callback(this[i], i, this)  
    );  
  }  
  
  return result;  
};
```

763. Implement filter

```
Array.prototype.myFilter = function (callback) {  
  const result = [];  
  
  for (let i = 0; i < this.length; i++) {  
    if (callback(this[i], i, this)) {  
      result.push(this[i]);  
    }  
  }  
}
```

```
    return result;
};
```

764. Implement reduce

```
Array.prototype.myReduce = function (
  callback,
  initialValue
) {
  let index = 0;
  let accumulator = initialValue;

  if (arguments.length < 2) {
    accumulator = this[0];
    index = 1;
  }

  for (; index < this.length; index++) {
    accumulator =
      callback(
        accumulator,
        this[index],
        index,
        this
      );
  }

  return accumulator;
};
```

A production-quality polyfill should carefully reproduce native edge-case behavior.

765. Flatten nested array

```
function flatten(arr) {
  return arr.reduce((result, value) => {
    return result.concat(
      Array.isArray(value)
        ? flatten(value)
        : value
    );
  }, []);
}
```

766. Group objects by property

```
function groupBy(items, key) {  
  return items.reduce((result, item) => {  
    const group = item[key];  
  
    (result[group] ??= []).push(item);  
  
    return result;  
  }, {});  
}
```

767. Simple Observable

Conceptual educational implementation:

```
class SimpleObservable {  
  constructor(subscribe) {  
    this.subscribe = subscribe;  
  }  
}  
  
const observable =  
  new SimpleObservable(observer => {  
    observer.next(1);  
    observer.next(2);  
    observer.complete();  
  });  
  
observable.subscribe({  
  next: value => console.log(value),  
  complete: () => console.log("done")  
});
```

Real RxJS Observables include significantly more behavior around teardown, errors, subscriptions and composition.

768. Simple EventEmitter

```
class EventEmitter {  
  constructor() {  
    this.listeners = new Map();  
  }  
}
```

```

on(event, handler) {
  if (!this.listeners.has(event)) {
    this.listeners.set(event, new Set());
  }

  this.listeners.get(event).add(handler);

  return () => {
    this.listeners.get(event)?.delete(handler);
  };
}

emit(event, value) {
  this.listeners
    .get(event)
    ?.forEach(handler => handler(value));
}
}

```

769. Memoization

```

function memoize(fn) {
  const cache = new Map();

  return function (arg) {
    if (cache.has(arg)) {
      return cache.get(arg);
    }

    const result = fn(arg);
    cache.set(arg, result);

    return result;
  };
}

```

For multiple/complex arguments, define a stable cache-key strategy.

770. Cache with expiration

```

class ExpiringCache {
  constructor() {
    this.cache = new Map();
  }
}

```

```

set(key, value, ttl) {
  this.cache.set(key, {
    value,
    expiresAt: Date.now() + ttl
  });
}

get(key) {
  const item = this.cache.get(key);

  if (!item) return undefined;

  if (Date.now() > item.expiresAt) {
    this.cache.delete(key);
    return undefined;
  }

  return item.value;
}

```

Quick revision

Two Sum	→ Map
Duplicates	→ Set
Frequency	→ Map
Binary Search	→ sorted + $O(\log n)$
Debounce	→ wait for quiet
Throttle	→ limit frequency
map	→ transform
filter	→ select
reduce	→ accumulate
Flatten	→ recursion / flat
GroupBy	→ reduce
Memoization	→ cache calculation
TTL cache	→ cache + expiry

Final Principal-Level Revision Sheet

The interview mental model

The uploaded question bank explicitly recommends preparing beyond the question itself:

QUESTION



CONCEPT



CODE



REAL-WORLD SCENARIO



ARCHITECTURAL TRADE-OFF

JavaScript

Closure

this

Prototype

Event Loop

Microtask/Macrotask

Promise

TypeScript

Interface vs Type

Generics

Union / Intersection

Type Guards / Narrowing

Utility Types

Conditional Types

Mapped Types

Structural Typing

RxJS

Observable

Subject / BehaviorSubject / ReplaySubject

Hot / Cold

map

switchMap

mergeMap

concatMap

exhaustMap

forkJoin

combineLatest

withLatestFrom

shareReplay
catchError
retry
subscription management

Angular

Standalone
Components
Input / Output / model
DI
Lifecycle
Forms
CVA
Routing
HTTP / Interceptors
Change Detection
OnPush
Signals
Control Flow
@defer
SSR / Hydration
Testing

Principal / Architecture

Enterprise architecture
State architecture
Performance
Security
RBAC
Microfrontends
Module Federation
Offline-first
IndexedDB
Metadata-driven UI
Dynamic components
API architecture
Observability
System design

One-line operator memory

```
map          → transform
switchMap    → latest
mergeMap     → parallel
concatMap    → sequential
exhaustMap   → ignore while busy
```

One-line state memory

```
Signal → local reactive state
RxJS   → asynchronous/event streams
NgRx   → complex shared application state
```

One-line Angular architecture

```
UI
↓
Facade / feature API
↓
State / domain services
↓
Data-access layer
↓
HTTP / IndexedDB
↓
Backend
```

One-line security

```
Frontend controls UX
Backend controls authorization
```

One-line performance

```
Measure → reduce work → reduce JS → reduce DOM → defer non-critical work
```

One-line Principal answer structure

When asked to design something:

1. Clarify requirements
2. Identify scale
3. Define boundaries
4. Define data flow
5. Define state ownership
6. Define API/contracts
7. Address performance
8. Address security
9. Address failure/recovery
10. Address observability
11. Explain trade-offs
12. Explain migration/deployment strategy

Enhanced Revision Layer

The original notes above remain the source-aligned Q1--Q770 revision material. The following is an additional study layer: **memory tricks, visual flows, race-condition analysis, and coding practice** mapped to the same 44 source topics.

2. JavaScript Async / Event Loop --- Enhancement

Memory trick: SMT = Synchronous → Microtasks → Tasks. Promise callbacks are microtasks; timers are tasks.

Flow diagram:

Sync code → Microtask queue → next Task

↑
Promise callbacks

Timer / DOM task → Task queue

Coding questions: 1. Predict output of nested Promise/setTimeout code. 2. Implement cancellable fetch with AbortController. 3. Prevent stale async results from overwriting newer results.

3. TypeScript Fundamentals --- Enhancement

Memory trick: interface/type = shape; union = OR; intersection = AND; unknown = safe unknown; any = escape hatch; never = impossible.

Coding questions: 1. Create a generic ApiResponse `<T> {=html}`. 2. Replace any with unknown + type guard. 3. Model success/error with a discriminated union.

4. Advanced TypeScript --- Enhancement

Memory trick: keyof = keys, typeof = value type, T[K] = indexed value type, Record = dictionary, Partial = optional, Pick = keep, Omit = remove.

Coding questions: 1. Implement DeepReadonly `<T> {=html}`. 2. Create a type-safe event map. 3. Extract function-valued properties with mapped types.

5. RxJS Fundamentals --- Enhancement

Memory trick: Observable = stream; next/error/complete = N/E/C. subscribe starts execution; unsubscribe stops subscription/teardown.

Coding questions: 1. Write a small production-style example for RxJS Fundamentals. 2. Debug a realistic failure involving RxJS Fundamentals.

3. Implement a reusable utility/component using RxJS Fundamentals and explain its complexity/trade-offs.

6. RxJS Transformation & Filtering --- Enhancement

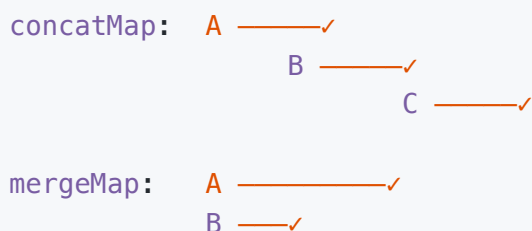
Memory trick: map = transform, filter = select, scan = running result, reduce = final result, tap = side effect.

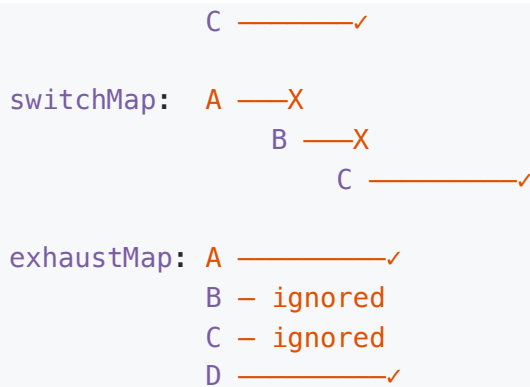
Coding questions: 1. Write a small production-style example for RxJS Transformation & Filtering. 2. Debug a realistic failure involving RxJS Transformation & Filtering. 3. Implement a reusable utility/component using RxJS Transformation & Filtering and explain its complexity/trade-offs.

7. RxJS Higher-Order Mapping --- Enhancement

Memory trick: switchMap = LATEST; mergeMap = PARALLEL; concatMap = QUEUE; exhaustMap = BUSY → IGNORE.

Flow diagram:





Coding questions: 1. Autocomplete with `debounceTime` + `switchMap`. 2. Sequential saves with `concatMap`. 3. Bounded-concurrency uploads with `mergeMap`.

8. RxJS Combination Operators --- Enhancement

Memory trick: `combineLatest` = latest from all; `forkJoin` = final from all; `zip` = position pairs; `withLatestFrom` = trigger + latest.

Coding questions: 1. Dashboard with `combineLatest`. 2. Parallel initial API loading with `forkJoin`. 3. Pair streams by position with `zip`.

9. RxJS Subjects / Multicasting --- Enhancement

Memory trick: `Subject` = broadcast; `BehaviorSubject` = current value; `ReplaySubject` = history; `AsyncSubject` = final value.

Coding questions: 1. Write a small production-style example for RxJS Subjects / Multicasting. 2. Debug a realistic failure involving RxJS Subjects / Multicasting. 3. Implement a reusable utility/component using RxJS Subjects / Multicasting and explain its complexity/trade-offs.

10. RxJS Error Handling --- Enhancement

Memory trick: `catchError` = recover; `retry` = try again; `throwError` = create error; `finalize` = always cleanup.

Coding questions: 1. Write a small production-style example for RxJS Error Handling. 2. Debug a realistic failure involving RxJS Error Handling. 3. Implement a reusable utility/component using RxJS Error Handling and explain its complexity/trade-offs.

11. RxJS Timing / Search --- Enhancement

Memory trick: `debounce` = wait for silence; `throttle` = limit rate; `audit` = sample at interval end.

Coding questions: 1. Write a small production-style example for RxJS Timing / Search. 2. Debug a realistic failure involving RxJS Timing / Search. 3. Implement a reusable utility/component using RxJS Timing / Search and explain its complexity/trade-offs.

12. Angular Fundamentals --- Enhancement

Memory trick: Component + Template + DI + Router. Standalone Angular = component-first composition.

Coding questions: 1. Write a small production-style example for Angular Fundamentals. 2. Debug a realistic failure involving Angular Fundamentals. 3. Implement a reusable utility/component using Angular Fundamentals and explain its complexity/trade-offs.

13. Angular Templates & Binding --- Enhancement

Memory trick: {{}} reads; [] sends data in; () sends events out; [()] combines both.

Coding questions: 1. Write a small production-style example for Angular Templates & Binding. 2. Debug a realistic failure involving Angular Templates & Binding. 3. Implement a reusable utility/component using Angular Templates & Binding and explain its complexity/trade-offs.

14. Angular Components & Communication --- Enhancement

Memory trick: Input = parent→child; Output = child→parent; service/state = cross-component; model = modern two-way binding.

Coding questions: 1. Build parent-child input/output table. 2. Build a component using model() two-way binding. 3. Implement sibling communication with a service.

15. Angular Lifecycle --- Enhancement

Memory trick: Create → Change → Content → View → Destroy.

Coding questions: 1. Write a small production-style example for Angular Lifecycle. 2. Debug a realistic failure involving Angular Lifecycle. 3. Implement a reusable utility/component using Angular Lifecycle and explain its complexity/trade-offs.

16. Angular Dependency Injection --- Enhancement

Memory trick: Token → Injector → Provider → Instance. useClass/create, useValue/fixed, useFactory/dynamic, useExisting/alias.

Coding questions: 1. Create InjectionToken configuration. 2. Implement useFactory provider. 3. Demonstrate component-scoped service instances.

17. Angular Directives --- Enhancement

Memory trick: Attribute = behavior; structural = view structure; component = directive with template.

Coding questions: 1. Write a small production-style example for Angular Directives. 2. Debug a realistic failure involving Angular Directives. 3. Implement a reusable utility/component using Angular Directives and explain its complexity/trade-offs.

18. Angular Template Primitives --- Enhancement

Memory trick: ng-container = grouping; ng-template = blueprint; ng-content = projection.

Coding questions: 1. Write a small production-style example for Angular Template Primitives. 2. Debug a realistic failure involving Angular Template Primitives. 3. Implement a reusable utility/component using Angular Template Primitives and explain its complexity/trade-offs.

19. ViewChild / Dynamic Components --- Enhancement

Memory trick: ViewChild finds view; ViewContainerRef hosts dynamic views; ComponentRef controls created instance.

Coding questions: 1. Write a small production-style example for ViewChild / Dynamic Components. 2. Debug a realistic failure involving ViewChild / Dynamic Components. 3. Implement a reusable utility/component using ViewChild / Dynamic Components and explain its complexity/trade-offs.

20. Angular Pipes --- Enhancement

Memory trick: Pure = input-driven; impure = frequent checks; AsyncPipe = subscribe/render/unsubscribe.

Coding questions: 1. Write a small production-style example for Angular Pipes. 2. Debug a realistic failure involving Angular Pipes. 3. Implement a reusable utility/component using Angular Pipes and explain its complexity/trade-offs.

21. Angular Forms --- Enhancement

Memory trick: Control → Group → Array. Value + status + touched/dirty are separate form state dimensions.

Flow diagram:

```
User input → FormControl → validators → VALID/INVALID/PENDING
```

Coding questions: 1. Registration form with sync validation. 2. Cross-field password validator. 3. Dynamic FormArray of addresses.

22. ControlValueAccessor --- Enhancement

Memory trick: W-C-T-D = writeValue, change callback, touched callback, disabled state.

Coding questions: 1. Build star-rating CVA. 2. Build custom dropdown CVA. 3. Support disabled state and touched state.

23. Angular Routing --- Enhancement

Memory trick: URL → route match → guards → resolver → component → outlet.

Coding questions: 1. Functional auth guard. 2. Unsaved-form canDeactivate guard. 3. Resolver for page bootstrap data.

24. Angular HTTP --- Enhancement

Memory trick: Component → service → HttpClient → interceptors → API → response/interceptors → state/UI.

Flow diagram:

```
Component → Service → HttpClient → Interceptor → API
                                     ↓
Component ← State/UI ← Interceptor ← Response
```

Coding questions: 1. Typed CRUD service. 2. Auth interceptor. 3. Request-count loading interceptor.

25. Angular Authentication & Security --- Enhancement

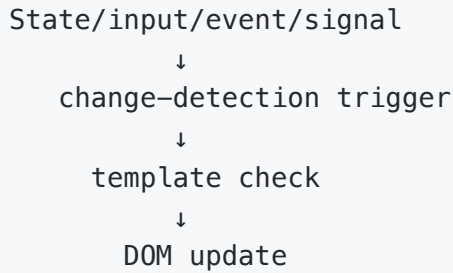
Memory trick: Authentication = who; authorization = what. Client guards improve UX; backend enforces security.

Coding questions: 1. RBAC guard + backend permission model. 2. Single-flight token refresh. 3. CSRF/XSS threat-model exercise.

26. Angular Change Detection --- Enhancement

Memory trick: OnPush = check on relevant triggers, not every possible mutation. Prefer immutable updates.

Flow diagram:



Coding questions: 1. Reproduce/fix OnPush mutation bug. 2. Optimize 10,000-row list. 3. Use ChangeDetectorRef deliberately.

27. Angular Signals --- Enhancement

Memory trick: signal = state; computed = derived; effect = side effect. Derive with computed, not effect.

Coding questions: 1. Signal-based shopping cart. 2. Replace simple BehaviorSubject store with Signals. 3. Bridge Observable ↔ Signal.

28. Angular Control Flow --- Enhancement

Memory trick: @if condition, @for iteration, @switch cases, @empty empty state, track identity.

Coding questions: 1. Write a small production-style example for Angular Control Flow. 2. Debug a realistic failure involving Angular Control Flow. 3. Implement a reusable utility/component using Angular Control Flow and explain its complexity/trade-offs.

29. Angular @defer --- Enhancement

Memory trick: Defer = load later. Trigger can be viewport, interaction, hover, idle, immediate, or when.

Coding questions: 1. Write a small production-style example for Angular @defer . 2. Debug a realistic failure involving Angular @defer . 3. Implement a reusable utility/component using Angular @defer and explain its complexity/trade-offs.

30. Angular Zone / Rendering --- Enhancement

Memory trick: Zone.js observes async activity; signals track dependencies; zoneless reduces reliance on global async patching.

Coding questions: 1. Write a small production-style example for Angular Zone / Rendering. 2. Debug a realistic failure involving Angular Zone / Rendering. 3. Implement a reusable utility/component using Angular Zone / Rendering and explain its complexity/trade-offs.

31. Angular Performance --- Enhancement

Memory trick: B-R-C-P = Bundle, Render, Compute, Paint. Optimize the actual bottleneck, not by guess.

Coding questions: 1. Write a small production-style example for Angular Performance. 2. Debug a realistic failure involving Angular Performance. 3. Implement a reusable utility/component using Angular Performance and explain its complexity/trade-offs.

32. Angular SSR / Hydration --- Enhancement

Memory trick: SSR creates HTML on server; hydration attaches client behavior; CSR creates UI in browser.

Coding questions: 1. Write a small production-style example for Angular SSR / Hydration. 2. Debug a realistic failure involving Angular SSR / Hydration. 3. Implement a reusable utility/component using Angular SSR / Hydration and explain its complexity/trade-offs.

33. Angular Testing --- Enhancement

Memory trick: Unit = one thing; integration = collaborating pieces; E2E = user journey.

Coding questions: 1. Write a small production-style example for Angular Testing. 2. Debug a realistic failure involving Angular Testing.

3. Implement a reusable utility/component using Angular Testing and explain its complexity/trade-offs.

34. NgRx --- Enhancement

Memory trick: Action → Reducer → Store → Selector → UI; Effect handles side effects/API. Reducers stay pure.

Flow diagram:

```
UI → Action → Reducer → Store → Selector → UI
      ↑
      Effect → API
```

Coding questions: 1. Complete CRUD feature. 2. Optimistic update with rollback. 3. Entity adapter + memoized selectors.

35. Angular Architecture --- Enhancement

Memory trick: Feature boundaries first; Core = infrastructure, Shared = reusable primitives, Feature = domain behavior.

Coding questions: 1. Write a small production-style example for Angular Architecture. 2. Debug a realistic failure involving Angular Architecture. 3. Implement a reusable utility/component using Angular Architecture and explain its complexity/trade-offs.

36. Dynamic / Plugin Architecture --- Enhancement

Memory trick: Metadata → validate → registry → resolve component → create → inputs/outputs → render.

Coding questions: 1. JSON-driven form renderer. 2. Component registry for metadata types. 3. Dynamic plugin with failure isolation.

37. Micro Frontends --- Enhancement

Memory trick: Host orchestrates; remote owns feature; contracts should be explicit; shared dependencies should be minimized.

Coding questions: 1. Host + remote module. 2. Remote failure fallback. 3. Explicit cross-MFE event contract.

38. Browser Internals --- Enhancement

Memory trick: DOM + CSSOM → Render Tree → Layout → Paint → Composite.

Coding questions: 1. Write a small production-style example for Browser Internals. 2. Debug a realistic failure involving Browser Internals. 3. Implement a reusable utility/component using Browser Internals and explain its complexity/trade-offs.

39. Web Fundamentals --- Enhancement

Memory trick: URL → DNS → TCP/TLS → HTTP → response → parse/cache/render.

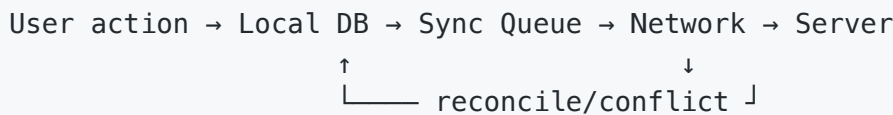
Coding questions: 1. Write a small production-style example for Web Fundamentals. 2. Debug a realistic failure involving Web Fundamentals.

3. Implement a reusable utility/component using Web Fundamentals and explain its complexity/trade-offs.

40. Offline-First / IndexedDB --- Enhancement

Memory trick: Local → Queue → Sync → Server → Reconcile. Design for disconnected operation first.

Flow diagram:



Coding questions: 1. IndexedDB repository. 2. Offline mutation queue. 3. Version-based conflict detection.

41. API / Backend Integration --- Enhancement

Memory trick: Pagination + filter + sort + search belong in explicit query state; protect against stale responses.

Coding questions: 1. Server pagination/filter/sort. 2. Debounced server search with stale-result protection. 3. Idempotent retryable command.

42. System Design / Principal-Level Angular --- Enhancement

Memory trick: Requirements → scale/constraints → boundaries → data flow → failure → security → observability → deployment.

Coding questions: 1. Write a small production-style example for System Design / Principal-Level Angular. 2. Debug a realistic failure involving System Design / Principal-Level Angular. 3. Implement a reusable utility/component using System Design / Principal-Level Angular and explain its complexity/trade-offs.

43. Design Patterns --- Enhancement

Memory trick: Factory=create; Strategy=choose behavior; Adapter=translate; Facade=simplify; Repository=persistence boundary; Observer=notify.

Coding questions: 1. Write a small production-style example for Design Patterns. 2. Debug a realistic failure involving Design Patterns.

3. Implement a reusable utility/component using Design Patterns and explain its complexity/trade-offs.

44. DSA / Coding --- Enhancement

Memory trick: HashMap=lookup; two pointers=ends; sliding window=contiguous range; stack=nested/previous; binary search=sorted search space.

Coding questions: 1. Two Sum. 2. Longest substring without repeating characters. 3. LRU Cache.

45. Most Important Questions for Your Target Role --- Enhancement

Memory trick: Remember the definition, execution flow, trade-off, and one real-world use case.

Coding questions: 1. Write a small production-style example for Most Important Questions for Your Target Role. 2. Debug a realistic failure involving Most Important Questions for Your Target Role. 3. Implement a reusable utility/component using Most Important Questions for Your Target Role and explain its complexity/trade-offs.

Race Conditions --- Dedicated Interview Chapter

What is a race condition?

A race condition happens when concurrent operations access or update shared state and the final result depends on which operation completes first.

1. Search race

```
"ang"      → Request A _____✓
"angular"  → Request B  _____✓

B renders first
A renders later → stale overwrite ❌
```

Solutions: `switchMap()`, `AbortController`, request IDs/sequence numbers, or server-side versioning.

2. Ordered save race

Save A _____✓
Save B _____✓

If B must be applied before A, concurrent execution is unsafe. Use `concatMap()` for client-side ordering and server-side version/idempotency protection for critical state.

3. Token refresh race

A ┌
B ┤ → 401 → many refresh calls ✗
C └

First 401 → one refresh → queue pending requests → replay after refresh ✓

4. Shared-state lost update

A reads 10 → writes 11
B reads 10 → writes 11
Expected 12, actual 11 ✗

Solutions: functional updates, transactions, atomic backend operations, optimistic locking/version checks.

Interview framework

Identify shared **state**
↓
Identify concurrent operations
↓
Define required ordering
↓
Cancel / serialize / deduplicate / coordinate
↓
Protect critical **state on** backend
↓
Test out-of-order completion